



الجمهورية العربية السورية  
المعهد العالي للعلوم التطبيقية والتكنولوجيا  
قسم النظم المعلوماتية  
اختصاص هندسة البرمجيات والذكاء الصناعي  
2026-2025

منصة دعم اتخاذ القرارات التقنية لفرق التطوير البرمجي بالاستناد إلى المحادثات  
الفورية وبدعم الذكاء الصناعي

A Platform for Supporting Technical Decision-Making in  
Software Development Teams Based on Real-Time Chat and  
Artificial Intelligence

إعداد  
سلمان محسن

إشراف  
د. غيداء ربدادي  
ما. عمار مخلوف

# المحتويات

|     |                                            |    |
|-----|--------------------------------------------|----|
| II  | قائمة الأشكال                              | 2  |
| III | الاختصارات                                 | 3  |
| IV  | المصطلحات                                  | 3  |
| 2   | الفصل الأول : المقدمة                      | 3  |
| 3   | الفصل الثاني : متطلبات المشروع             | 3  |
| 3   | 1.2 مقدمة                                  | 3  |
| 3   | 2.2 المتطلبات الوظيفية                     | 5  |
| 5   | 3.2 المتطلبات غير الوظيفية                 | 6  |
| 6   | 4.2 تحليل المتطلبات                        | 13 |
| 13  | الفصل الثالث : تصميم النظام                | 13 |
| 13  | 1.3 مقدمة                                  | 13 |
| 13  | 2.3 الأنماط المعمارية والتصميمية           | 14 |
| 14  | 3.3 مخطط مكونات النظام (Component Diagram) | 15 |
| 15  | 4.3 تصميم قاعدة البيانات (ERD)             | 17 |
| 17  | 5.3 مخططات التتالي التصميمية               | 20 |
| 20  | الفصل الرابع : تنفيذ النظام                | 20 |
| 20  | 1.4 مقدمة                                  | 20 |
| 20  | 2.4 البيئات والأدوات المستخدمة             | 22 |
| 22  | 3.4 تنفيذ مكونات النظام                    | 31 |
| 31  | الفصل الخامس : خطة اختبار النظام           | 31 |
| 31  | 1.5 مقدمة                                  | 31 |
| 31  | 2.5 استراتيجية الاختبار العامة             | 31 |
| 31  | 3.5 اختبارات الوحدة                        | 32 |
| 32  | 4.5 اختبارات التكامل                       | 33 |
| 33  | 5.5 اختبار الانحدار                        | 34 |
| 34  | المراجع                                    |    |

## قائمة الأشكال

|    |                                                                        |     |
|----|------------------------------------------------------------------------|-----|
| 7  | مخطط حالات الاستخدام لمنصة Synapse . . . . .                           | 1.2 |
| 8  | مخطط التتالي لعملية تفعيل التذكرة وتوليد خطة التنفيذ . . . . .         | 2.2 |
| 9  | مخطط التتالي لعملية اعتماد الخطة وتنفيذ وكيل الترميز . . . . .         | 3.2 |
| 10 | مخطط التتالي لعملية إنشاء تذكرة عبر GitHub . . . . .                   | 4.2 |
| 11 | مخطط التتالي لعملية إغلاق التذكرة تلقائياً عند دمج طلب الدمج . . . . . | 5.2 |
| 12 | مخطط التتالي لعملية ربط المشروع بمستودع GitHub . . . . .               | 6.2 |
| 15 | مخطط مكونات النظام (UML Component Diagram) . . . . .                   | 1.3 |
| 16 | مخطط الكيانات والعلاقات — النطاق الأساسي . . . . .                     | 2.3 |
| 17 | مخطط الكيانات والعلاقات — تنفيذ الوكلاء والتكامل . . . . .             | 3.3 |
| 18 | مخطط التتالي التصميمي لعملية توليد خطة التنفيذ . . . . .               | 4.3 |
| 19 | مخطط التتالي التصميمي لعملية تنفيذ خطة الترميز . . . . .               | 5.3 |

# الاختصارات

| Acronym | Full Form                      | الترجمة العربية            |
|---------|--------------------------------|----------------------------|
| ERD     | Entity-Relationship Diagram    | مخطط الكيانات والعلاقات    |
| LLM     | Large Language Model           | نموذج لغوي كبير            |
| RAG     | Retrieval-Augmented Generation | التوليد المعزّز بالاسترجاع |
| UML     | Unified Modeling Language      | لغة النمذجة الموحدة        |

# المصطلحات

**نموذج لغوي كبير (Large Language Model):** شبكة عصبونية مُصممة لفهم وتوليد والرد على النصوص الطبيعية وهي مدربة على كمية هائلة من البيانات النصية 1

**وكيل الذكاء الاصطناعي (AI Agent):** كيان ذكي قادر على اتخاذ قرارات وتنفيذ أعمال بشكل مستقل 1, 2

# الخلاصة

لا تزال فرق التطوير البرمجي تعتمد على منصات تواصل فوري لمناقشة المشكلات التقنية والوصول إلى قرارات جماعية حولها، ثم يتوجب على أحد المطورين لاحقاً إعادة صياغة خلاصة هذا النقاش يدوياً كوصف نصي يُقدّم لأداة وكيل الذكاء الصناعي في بيئة منفصلة تماماً، وهي خطوة وسيطة يدوية تتكرر مع كل مهمة وتُفقد جزءاً من سياق النقاش الأصلي. تعالج منصة «Synapse» هذه الفجوة بدمج طبقة التواصل الجماعي بين فرق التطوير مع نظام وكلاء آليين يعمل مباشرة على سياق النقاش، وفق مبدأ ناظم: **الإنسان يتداول، والوكيل ينفذ، والإنسان يراجع.**

يتوزع النظام على خدمة خلفية أساسية بمعمارية Modular Monolith مسؤولة عن كامل النماذج والبيانات، وخدمتين مصغرتين مستقلتين: خدمة التخطيط التي تحوّل خلاصة النقاش، بعد تفعيلها من قائد الفريق، إلى خطة تنفيذ مفصلة بالاستعانة بالتوليد المعزّز بالاسترجاع (Retrieval-Augmented Generation - RAG) على سياق المشروع، وخدمة الترميز التي تنفّذ هذه الخطة خطوة بخطوة داخل حاوية معزولة مؤقتة، وصولاً إلى فتح طلب سحب جاهز للمراجعة على منصة استضافة الرماز. يعتمد النظام حصراً على نموذج لغوي كبير مفتوح الوزن يعمل محلياً عبر محرك استدلال عالي الأداء دون أي اعتماد على واجهات برمجية خارجية للذكاء الصناعي، وهو قيد فرضته محدودية العتاد المتاح، وطُبِّقت حوله عدة آليات حماية تشمل الفصل بين المحتوى الموثوق وغير الموثوق، وحماية المسارات الحساسة، وبوابة تحقق من نطاق المهمة، وقفلاً موزعاً يمنع تعارض الوكلاء على الملف الواحد. يتلخص إسهام المشروع في نموذج بيانات موحد يجمع المحادثة والمهام وسجل تنفيذ الوكيل ضمن كيان واحد، مقروناً ببنية أدوار وموافقات بشرية مدمجة في صلب دورة التطوير بدل أن تكون طبقة لاحقة عليها.

## Abstract

Software teams still rely on real-time chat platforms to discuss technical problems and reach a shared decision, after which a developer must manually restate that discussion as a text prompt for an AI coding tool running in a completely separate environment -- a manual step repeated for every task that loses part of the original context. Synapse closes this gap by merging the team's communication layer with an agentic system that operates directly on the discussion context, under one governing principle: **humans deliberate, the agent executes, humans review.**

The system is split into a core backend service built as a **Modular Monolith**, and two independent microservices: a Planning Service that turns an activated discussion thread into a detailed implementation plan using Retrieval-Augmented Generation over the project's own context, and a Coding Service that executes that plan step by step inside an ephemeral sandboxed container, ending in a ready-to-review pull request. The system relies exclusively on a locally-served open-weight Large Language Model, with no external AI APIs, a constraint driven by limited available hardware, and is guarded by several safeguards including untrusted-content isolation, protected-path checks, a task-scope gate, and distributed locking to prevent concurrent agents from conflicting on the same file.

The project's contribution is a unified data model that combines chat, tickets, and agent execution history within one entity, paired with a role-and-approval structure built into the core of the development cycle rather than layered on top of it.

# الفصل الأول

## المقدمة

أحدث الذكاء الصناعي في السنوات الأخيرة تحولاً واسعاً طال معظم المجالات العملية، من الطب والتصنيع إلى التعليم والخدمات المالية، حيث أصبحت النماذج اللغوية الكبيرة (LLM) وأنظمة وكيل الذكاء الصناعي قادرة على تنفيذ مهام كانت حتى وقت قريب حكراً على الخبرة البشرية. يكمن جوهر هذا التأثير في قدرة هذه الأنظمة على أتمتة الإجراءات المتكررة والرتيبة التي تستهلك وقتاً وجهداً بشرياً كبيرين دون أن تتطلب بالضرورة إبداعاً معقداً، مما يوفر وقتاً وموارد يصعب تحقيقهما بالطرق التقليدية المعتمدة كلياً على العمل اليدوي.

هندسة البرمجيات ليست استثناءً من هذا التحول، بل تُعد من أكثر المجالات التي استفادت من قدرات الذكاء الصناعي التوليدي؛ فالنماذج اللغوية المُدرَّبة خصيصاً على الرماز البرمجي أصبحت قادرة على كتابة رماز صحيح وظيفياً بالاعتماد على وصف نصي مقتضب. وتطورت هذه القدرة من مجرد أدوات إكمال تلقائي للرماز، إلى وكلاء آليين أكثر استقلالية قادرين على تنفيذ مهام تطوير كاملة من الفهم إلى الاختبار. تكمن قيمة هذه الأدوات في أتمتتها للخطوات الرتيبة من دورة التطوير، كتابة الشيفرة المتكررة، تشغيل الاختبارات، وإصلاح الأخطاء البسيطة بسرعة تفوق بمراحل ما يستغرقه التنفيذ البشري اليدوي البحث لنفس المهام.

إلا أن هذه الأدوات، على الرغم من قدراتها، تعمل بمعزل عن السياق الذي يُتخذ ضمنه القرار التقني فعلياً؛ فالفرق البرمجية لا تزال تعتمد على منصات تواصل فوري لمناقشة المشكلات والوصول إلى قرارات جماعية حول كيفية الحل، ثم يتوجب على أحد المطورين لاحقاً إعادة صياغة خلاصة هذا النقاش يدوياً على شكل وصف نصي يُقدّم لأداة الذكاء الصناعي في بيئة منفصلة تماماً. هذه الخطوة الوسيطة يدوية بطبيعتها، تتكرر مع كل مهمة جديدة، وتُفقد جزءاً من السياق الأصلي للنقاش، رغم أن الأدوات المتاحة قادرة تقنياً على أتمتة ما بعدها.

يهدف هذا المشروع إلى معالجة هذه الفجوة عبر تطوير منصة ذكية باسم «Synapse»، تدمج طبقة التواصل الجماعي بين فرق التطوير مع نظام وكلاء آليين يعمل مباشرة على سياق النقاش دون حاجة لإعادة صياغته يدوياً. يتولى وكيل تخطيط تحويل خلاصة النقاش، بعد تفعيلها من قبل قائد الفريق، إلى خطة تنفيذ مفصلة قابلة للمراجعة البشرية، ثم يتولى وكيل ترميز تنفيذ هذه الخطة خطوة بخطوة داخل بيئة معزولة، وصولاً إلى فتح طلب سحب جاهز للمراجعة على منصة GitHub. وبذلك تستفيد المنصة من قدرة الذكاء الصناعي على أتمتة الإجراءات المتكررة لتوفير الوقت، مع الحفاظ في الوقت نفسه على مبدأ ناظم صريح: الإنسان يتداول، والوكيل ينفذ، والإنسان يراجع، بحيث لا يتقدّم أي إجراء آلي دون قرار بشري سابق له عند كل نقطة حرجية من دورة التطوير.

# الفصل الثاني

## متطلبات المشروع

### 1.2 مقدمة

تُعد الدراسة التحليلية المرحلة الانتقالية التي تُترجم فيها الأهداف النظرية والفجوات البحثية التي تمت مناقشتها في الفصل السابق إلى مواصفات نظامية قابلة للتنفيذ. تهدف هذه الدراسة إلى تحديد وتوثيق متطلبات منصة «Synapse» بشكل دقيق، بما يضمن بناء معمارية قادرة على سد الفجوة بين المحادثات الفورية لفرق التطوير والتنفيذ الآلي عبر وكلاء الذكاء الاصطناعي. تُنظم هذه الدراسة حول محورين أساسيين يحددان هوية النظام وسلوكه:

- **المتطلبات الوظيفية (Functional Requirements):** والتي تُعرّف الخدمات والوظائف الجوهرية التي يجب أن يقدمها النظام، مثل مراقبة المحادثات، اكتشاف الإجماع التقني، وتنسيق العمل مع وكلاء الذكاء الاصطناعي ومنصات التحكم بالمصدر.
- **المتطلبات غير الوظيفية (Non-Functional Requirements):** والتي تحدد معايير الجودة، الأداء، الأمان، وقابلية التوسع التي تضمن موثوقية النظام في بيئات التطوير الحقيقية.

ولتوضيح التفاعلات المعقدة بين المستخدمين، ووكلاء الذكاء الاصطناعي، والأنظمة الخارجية (مثل GitHub)، يتم تبني هذه الدراسة التي تعتمد على نمذجة سلوك النظام باستخدام لغة النمذجة الموحدة (Unified Modeling Language - UML) من خلال بعض المخططات مثل:

- **مخطط حالات الاستخدام (Use Case Diagram):** يحدد نطاق النظام من خلال رسم الحدود بين الفاعلين Actors. سواء كانوا مطورين، أو وكلاء آليين، أو أنظمة خارجية والوظائف الرئيسية التي يوفرها النظام لكل فاعل
- **مخطط التتالي (Sequence Diagram):** يوضح التسلسل الزمني الدقيق للرسائل والتفاعلات بين مكونات النظام لتنفيذ سيناريوهات معينة (مثل تحويل قرار تقني في المحادثة إلى Pull Request).

يستعرض هذا الفصل في الأقسام التالية المتطلبات الوظيفية، ثم المتطلبات غير الوظيفية، يليها مخططات حالات الاستخدام، وأخيراً مخططات التتالي للسيناريوهات الرئيسية التي توضح دورة حياة القرار التقني داخل المنصة من لحظة طرحه في المحادثة وحتى تنفيذه.

### 2.2 المتطلبات الوظيفية

يستعرض هذا القسم المتطلبات الوظيفية لمنصة «Synapse»

#### 1.2.2 إدارة الحسابات والمصادقة

1. يجب أن يسمح النظام للمستخدم بإنشاء حساب أو تسجيل الدخول عبر البريد الإلكتروني وكلمة المرور.
2. يجب أن يسمح النظام بتسجيل الدخول أو إنشاء الحساب عبر منصتي GitHub أو Google.
3. يجب أن يسمح النظام للمستخدم المسجل في قاعدة البيانات بربط حساب GitHub أو Google بحسابه الحالي.
4. يجب أن يوفر النظام واجهة للبحث عن المستخدمين بالاسم أو البريد الإلكتروني، لاستخدامها في إرسال الدعوات إلى مساحات العمل والمشاريع والقنوات.



## 2.2.2 إدارة مساحات العمل والمشاريع والقنوات

1. يجب أن يسمح النظام بإنشاء مساحة عمل (Workspace) وعرضها وتعديلها وحذفها، وعرض قائمة مساحات العمل التي ينتمي إليها المستخدم.
2. يجب أن يسمح النظام بإدارة أعضاء مساحة العمل، بما في ذلك ترقية عضو إلى مالك (Owner) وإزالة عضو منها.
3. يجب أن يسمح النظام بإنشاء مشروع (Project) ضمن مساحة عمل معينة، مرتبطاً بمستودع GitHub واحد، وعرض المشاريع وتعديلها وحذفها.
4. يجب أن يسمح النظام بإدارة أعضاء المشروع وأدوارهم (قائد الفريق، عضو، مراقب)، بما يشمل الإضافة والتعديل والإزالة.
5. يجب أن يسمح النظام بإنشاء قناة (Channel) ضمن مشروع معين، وعرض القنوات وتعديلها وحذفها.
6. يجب أن يوفر النظام إنشاءً خاصاً لقناة مخصصة لقادة الفريق (Leads Channel) ضمن المشروع.
7. يجب أن يسمح النظام بإدارة أعضاء القناة وأدوارهم، بما يشمل الإضافة والتعديل والإزالة.

## 3.2.2 إدارة التذاكر

1. يجب أن يسمح النظام لعضو القناة بإنشاء تذكرة (Ticket) ضمن قناة معينة، وعرض قائمة التذاكر الخاصة بها.
2. يجب أن يوفر النظام عرضاً تفصيلياً للتذكرة، يتضمن بيانات التذكرة، والرسائل، وحالة النقاش (Thread State).
3. يجب أن يسمح النظام بتعديل بيانات التذكرة (العنوان، الوصف، الأولوية).
4. يجب أن يسمح النظام بترحيل التذكرة (Route) من قناة إلى قناة أخرى ضمن المشروع نفسه.
5. يجب أن يسمح النظام بتفعيل التذكرة لنقلها من حالة الانتظار إلى حالة النقاش الفعلي.
6. يجب أن يسمح النظام بتوليد خطة تنفيذ آلية (Generate Plan) للتذكرة، عبر تفعيل وكيل التخطيط (Planning Agent).
7. يجب أن يسمح النظام بتقسيم التذكرة إلى تذكرتين فرعيتين أو أكثر، عند تجاوز نطاقها حجماً يصعب تنفيذه ضمن تدفق عمل واحد.
8. يجب أن يسمح النظام بإغلاق التذكرة يدوياً عند اكتمال العمل عليها أو عدم الحاجة إليها.

## 4.2.2 نظام المحادثة الفورية

1. يجب أن يسمح النظام لأعضاء التذكرة بإرسال رسائل نصية ضمن سلسلة النقاش الخاصة بها.
2. يجب أن يسمح النظام بعرض الرسائل المرسلّة من قِبَل المطوّرين والنظام والوكلاء.
3. يجب أن يسمح النظام بتعديل رسالة تم إرسالها مسبقاً.
4. يجب أن يسمح النظام بحذف رسالة بحيث يُحتفظ بالمحتوى في قاعدة البيانات لأغراض التدقيق، بينما يُخفى عن واجهات العرض.

## 5.2.2 نظام الدعوات والصندوق الوارد

1. يجب أن يوفر النظام صندوقاً وارداً موحداً (Inbox) لكل مستخدم، يجمع الدعوات والإشعارات في قائمة واحدة.
2. يجب أن يسمح النظام للمستخدمين بقبول الدعوة أو رفضها.
3. يجب أن يسمح النظام بإرسال دعوة انضمام مستهدفة (بتحديد المستخدم والدور) على مستوى مساحة العمل، أو المشروع، أو القناة.

## 6.2.2 التكامل مع GitHub

1. يجب أن يسمح النظام لقائد الفريق بربط مشروع بمستودع GitHub عبر GitHub App، وعرض بيانات هذا الربط أو إزالته لاحقاً.

## 7.2.2 تنفيذ الوكلاء الآليين

1. يجب أن يسمح النظام لأي عضو في المشروع بعرض تفاصيل الخطة الحالية وحالتها.
2. يجب أن يسمح النظام لقائد الفريق أو مالك المشروع، دون سواهما، باعتماد الخطة المولدة أو رفضها، وذلك تبعاً لسياسة الاعتماد المحددة عند إنشاء القناة.
3. يجب أن يسمح النظام لقائد الفريق أو مالك المشروع بتعديل محتوى الخطة (Plan) قبل اعتمادها.
4. يجب أن يُطلق النظام، بعد اعتماد الخطة، مهام تنفيذ آلية عبر وكيل التنفيذ (Code Agent)، تشمل إنشاء فرع (Branch)، تنفيذ التغييرات ضمن حاوية معزولة، وفتح طلب دمج (Pull Request) عند الانتهاء.

## 3.2 المتطلبات غير الوظيفية

في حين تُحدد المتطلبات الوظيفية الخدمات التي يقدمها النظام، تصف المتطلبات غير الوظيفية الخصائص النوعية التي يجب أن تتحلى بها هذه الخدمات لتكون موثوقة وقابلة للاستخدام في سياق فرق تطوير حقيقية، رغم القيود العملية لبيئة النظام.

### 1.3.2 الأداء (Performance)

1. يجب أن يعتمد النظام البث اللحظي باستخدام WebSocket لإيصال الرسائل والتحديثات إلى المستخدمين، بدلاً من الاستقصاء الدوري (Polling).
2. يجب أن تُنفَّذ المهام الثقيلة المعتمدة على الذكاء الاصطناعي (توليد الخطط، تنفيذ الرماز) بشكل غير متزامن عبر طابور مهام مخصص، بحيث لا يُحجب المستخدم أثناء معالجتها.

### 2.3.2 الأمان (Security)

1. يجب ألا تتجاوز صلاحية أي رمز تثبيت (Installation Token) صادر عن GitHub App ساعة واحدة من الزمن، ويجب عدم تخزين أي رمز من هذا النوع بعد انتهاء استخدامه الفوري.
2. يجب أن يتحقق النظام من توقيع كل حدث Webhook وارد من GitHub قبل معالجته، لضمان أن مصدر الحدث موثوق.
3. يجب أن تُطبَّق الصلاحيات على مستوى كل مشروع على حدة، بحيث لا يملك أي مستخدم صلاحية تتجاوز دوره المحدد ضمن ذلك المشروع تحديداً.

### 3.3.2 الموثوقية والتوفر (Reliability & Availability)

1. يجب أن يُعاد تنفيذ أي خطوة فشلت لأسباب تقنية بحتة (Hard Technical Failure) عبر 3 محاولات كحد أقصى، بفواصل زمنية متصاعدة قدرها دقيقة واحدة، ثم 5 دقائق، ثم 15 دقيقة.
2. يجب ألا يُفقد أي خطوة منجزة عند إعادة المحاولة؛ بل يُستأنف التنفيذ من نقطة التوقف (Checkpoint Resume) دون تكرار ما أنجز فعلاً.
3. يجب ألا يتجاوز عدد محاولات تصحيح الوكيل لخطأ ناتج عن سلوك غير صحيح للنموذج (Soft AI Failure) محاولتين، يتم بعدهما نقل الحالة إلى مراجعة المطور.
4. يجب ألا يتجاوز إجمالي عدد محاولات التنفيذ ضمن عملية تشغيل واحدة للوكيل الآلي (بجميع أنواع الفشل مجتمعة) 5 محاولات، لمنع الدخول في حلقة تكرار (Stuck Loop).
5. يجب أن يُحفظ سجل تدقيق كامل لكل خطوة من خطوات تنفيذ الوكيل بشكل مستقل عن نجاح أو فشل العملية الكلية.

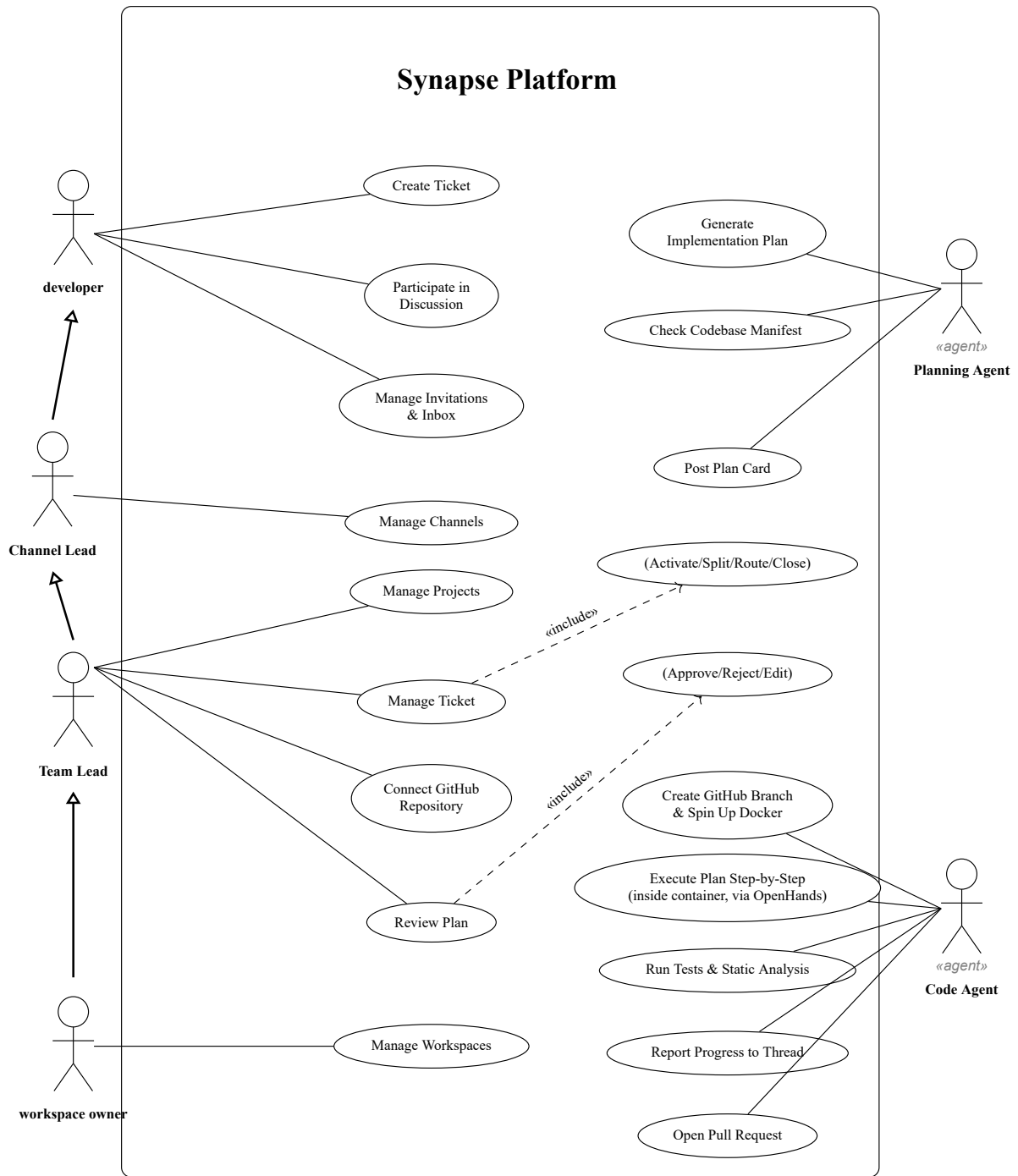
## 4.2 تحليل المتطلبات

### 1.4.2 مخطط حالات الاستخدام

يوضح الشكل 1.2 مخطط حالات الاستخدام (Use Case Diagram) لمنصة «Synapse». تم استبعاد حالة استخدام «تسجيل الدخول» من المخطط بوصفها إجراءً تمهيدياً (Precondition) لازماً لأي تفاعل مع النظام، وليست حالة استخدام جوهرية تميز سلوك المنصة عن غيرها.

#### الفاعلون (Actors)

- **المطور (Developer):** الفاعل الأساسي، يملك صلاحيات إنشاء التذاكر والمشاركة في النقاش وإدارة الدعوات الخاصة به.
- **قائد القناة (Channel Lead):** فاعل مسؤول عن إدارة قناة معينة، يملك صلاحيات أعلى من المطور، ويُضاف إليها صلاحيات إدارة أعضاء القناة.
- **قائد الفريق (Team Lead):** يرث جميع صلاحيات العضو عبر علاقة تعميم (Generalization)، لذلك لا تتكرر حالات استخدام العضو معه في المخطط؛ ويُضيف إليها صلاحيات إدارية إضافية (إدارة المساحات والمشاريع والقنوات، تفعيل التذاكر، مراجعة الخطط، وربط المشروع بـ GitHub).
- **مسؤول مساحة العمل (Workspace Owner):** فاعل مسؤول عن إدارة مساحة العمل، يملك صلاحيات أعلى من قائد الفريق، ويُضاف إليها صلاحيات إدارة أعضاء مساحة العمل.
- **وكيل التخطيط (Planning Agent):** فاعل غير بشري مسؤول عن توليد خطة عمل.
- **وكيل برمجي (Coding Agent):** فاعل غير بشري مسؤول عن تنفيذ الخطة التي سبَق توليدها من قِبَل وكيل التخطيط.



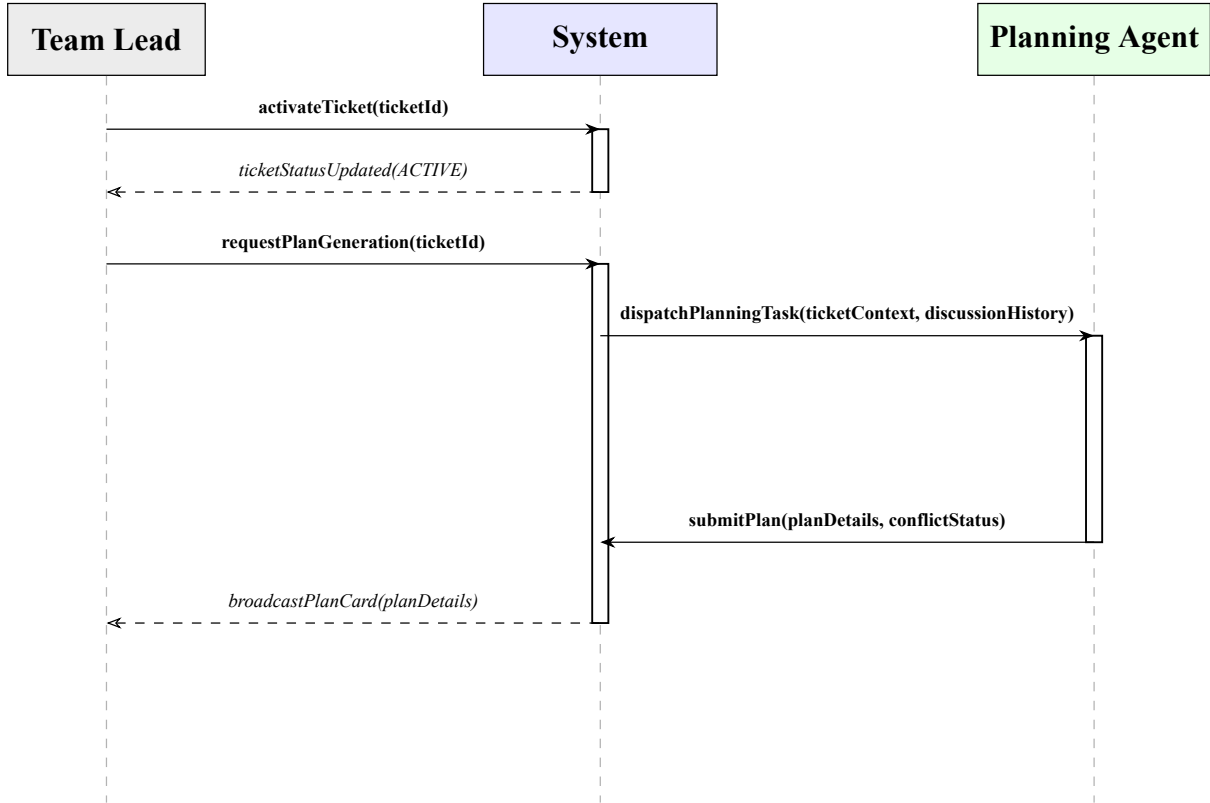
الشكل 1.2: مخطط حالات الاستخدام لمنصة Synapse

## 2.4.2 مخططات التتالي

تجدر الإشارة إلى أنه لم يتم رسم مخطط تتالي لكل سيناريو ممكن ضمن النظام، إذ إن ذلك يخرج عن نطاق هذه الدراسة ولا يضيف قيمة تحليلية تُذكر لكثير من العمليات الروتينية (كعمليات الإنشاء والعرض والتعديل والحذف الاعتيادية على الكيانات مثل مساحات العمل والمشاريع والقنوات والرسائل). بل تم الاختصار على السيناريوهات الأساسية التي تمثل المسار الأساسي لفكرة المنصة، وتحديداً تلك المرتبطة بدورة حياة القرار التقني من لحظة طرحه في النقاش وحتى تنفيذه آلياً عبر الوكلاء، إضافة إلى سيناريوهات التكامل مع GitHub بوصفها الأكثر تعقيداً من الناحية التفاعلية والأجدر بالتوثيق.

### مخطط التتالي: تفعيل التذكرة وتوليد خطة التنفيذ

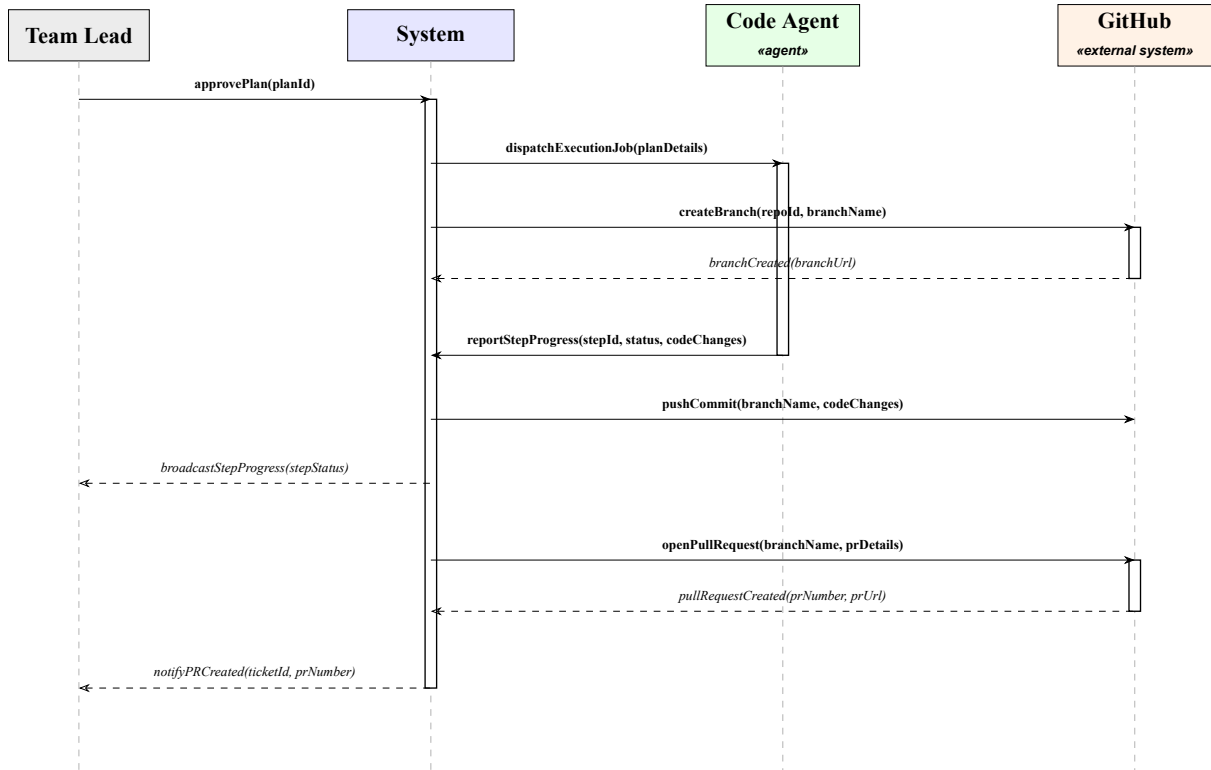
يوضح الشكل 2.2 مخطط التتالي لعملية تفعيل التذكرة وتوليد خطة التنفيذ الآلية عبر وكيل التخطيط (Planning Agent).



الشكل 2.2: مخطط التتالي لعملية تفعيل التذكرة وتوليد خطة التنفيذ

### مخطط التتالي: اعتماد الخطة وتنفيذ وكيل الترميز

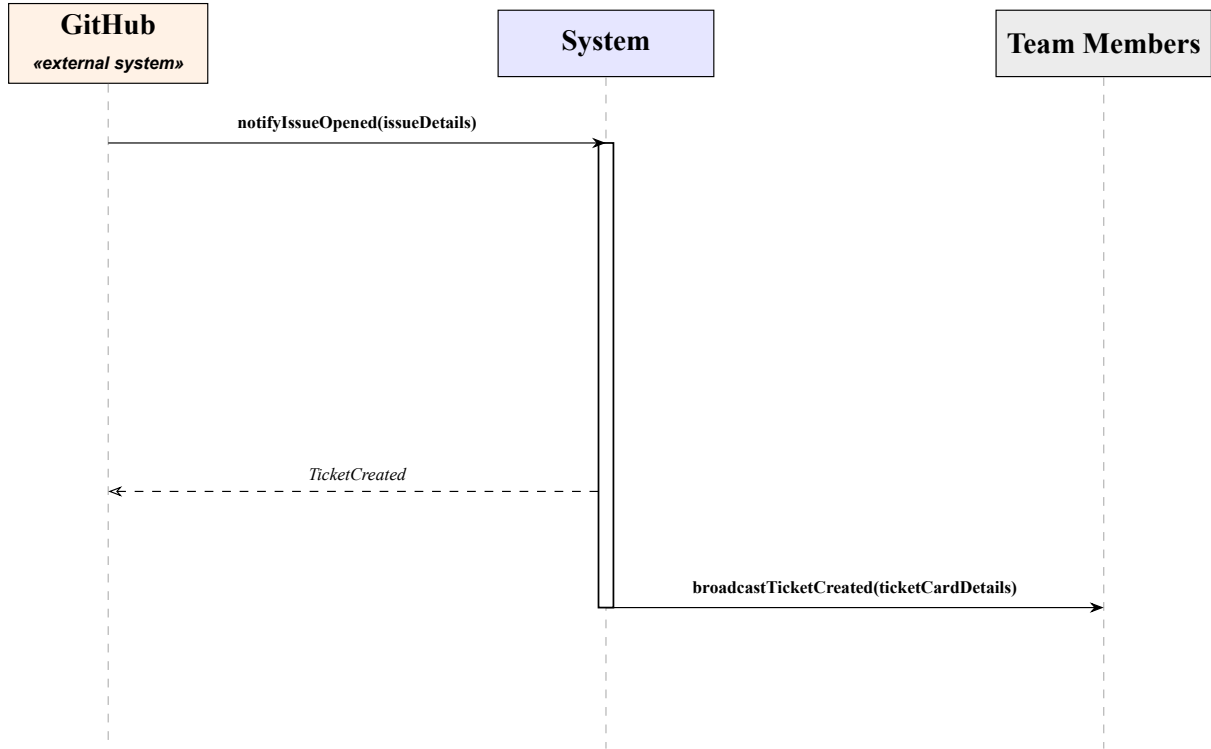
يوضح الشكل 3.2 مخطط التتالي لعملية اعتماد الخطة وتنفيذها آلياً، حيث يتم تمثيل المنصة كحد نظامي مغلق (System) يتفاعل مع قائد الفريق كفاعل أساسي، وكل من وكيل الترميز و GitHub كفاعلين خارجيين فرعيين.



الشكل 3.2: مخطط التتالي لعملية اعتماد الخطة وتنفيذ وكيل الترميز

### مخطط التتالي: إنشاء تذكرة تلقائياً من خلال GitHub Issue

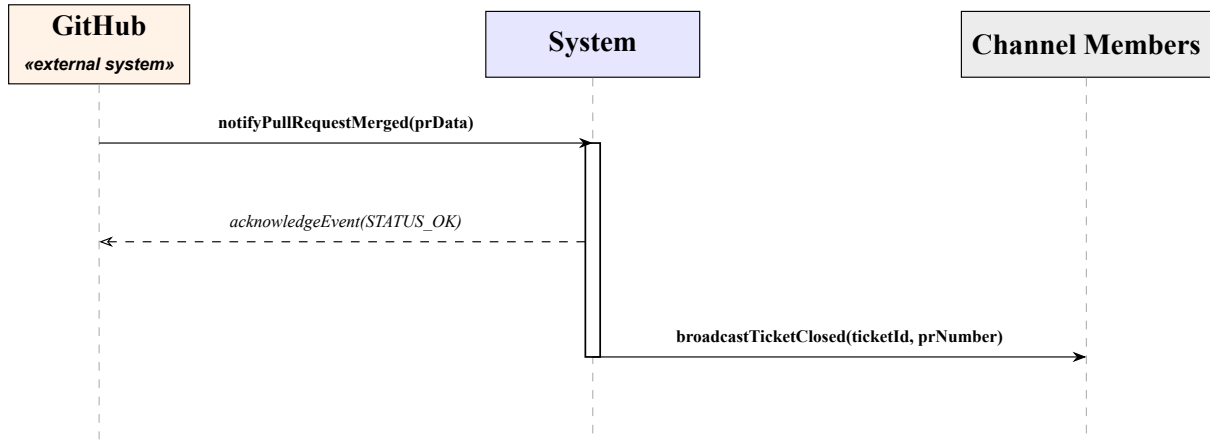
يوضح الشكل 4.2 مخطط التتالي المفهومي (SSD) لعملية إنشاء تذكرة تلقائياً في المنصة لإنشاء قضية جديدة (Issue) في GitHub.



الشكل 4.2: مخطط التتالي لعملية إنشاء تذكرة عبر GitHub

### مخطط التتالي: إغلاق التذكرة تلقائياً عند دمج طلب الدمج (Pull Request)

يوضح الشكل 5.2 مخطط التتالي المفهومي (SSD) لعملية التحديث التلقائي لحالة التذكرة وإغلاقها فور استقبال إشعار دمج طلب الدمج (Pull Request) من GitHub.

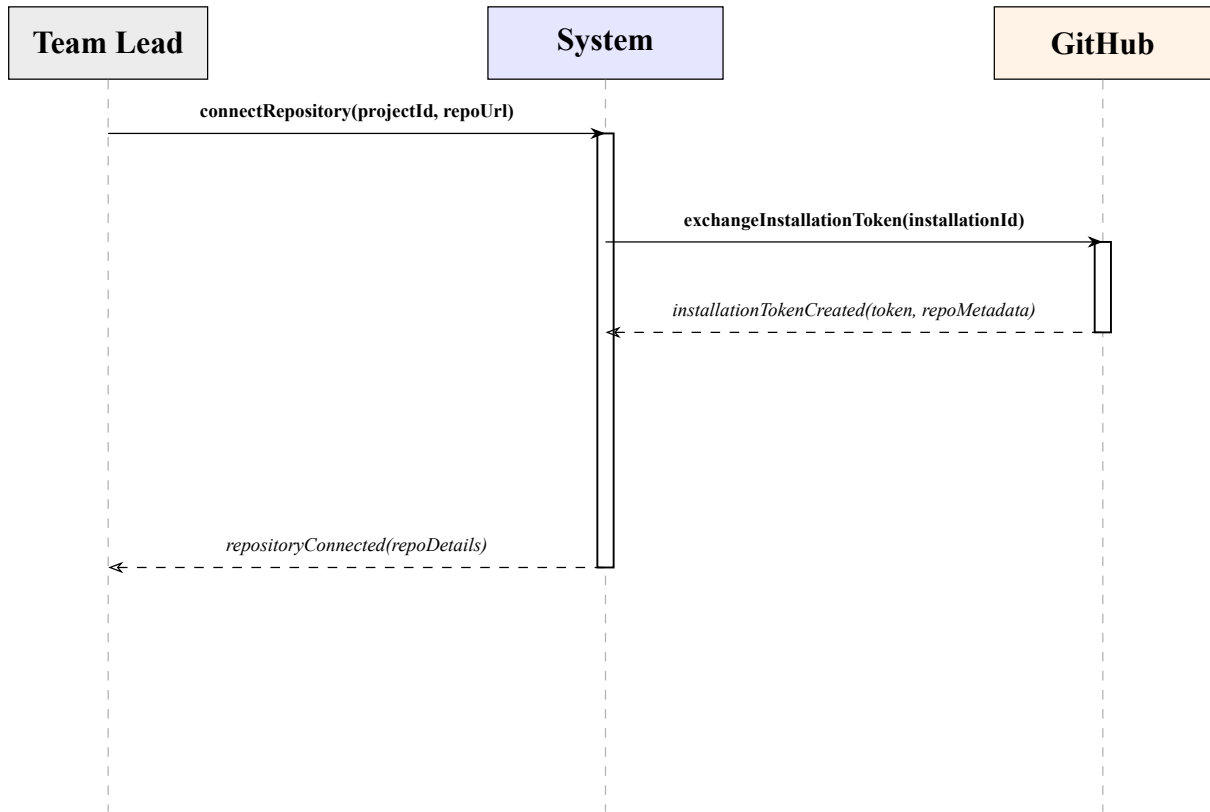


الشكل 5.2: مخطط التتالي لعملية إغلاق التذكرة تلقائياً عند دمج طلب الدمج



### مخطط التتالي: ربط المشروع بمستودع GitHub

يوضح الشكل 6.2 مخطط التتالي المفهومي (SSD) لعملية ربط مشروع محدد ضمن المنصة بمستودع GitHub عبر تطبيق GitHub App.



الشكل 6.2: مخطط التتالي لعملية ربط المشروع بمستودع GitHub

# الفصل الثالث

## تصميم النظام

### 1.3 مقدمة

في الفصل السابق تم تناول النظام من منظور تحليلي، حيث عُومل النظام ككيان واحد متكامل (black box) في مخططات تنالي النظام (SSD) دون الخوض في تفاصيل تنفيذه الداخلي. يهدف هذا الفصل إلى الانتقال من التحليل إلى التصميم، من خلال تفكيك هذا الكيان إلى مكوناته البرمجية الفعلية، وتوضيح كيفية تفاعلها فيما بينها لتحقيق المتطلبات الموثقة سابقاً. يغطي هذا الفصل أربعة محاور تصميمية رئيسية:

- **الأنماط المعمارية والتصميمية:** يحدد الهيكل المعماري الكلي للنظام والأنماط التصميمية المتبعة لتنظيم بنيته البرمجية وعلاقاتها المتبادلة.
- **مخطط مكونات النظام (Component Diagram):** يوضح المكونات البرمجية للنظام، وواجهاتها المُقدَّمة، واعتماد كل مكون على غيره من المكونات.
- **تصميم قاعدة البيانات (مخطط الكيانات والعلاقات (Entity-Relationship Diagram - ERD):** يوضح الكيانات المخزنة في قاعدة البيانات العلائقية والعلاقات بينها بالتوافق التام مع البنية البرمجية والبروتوكولات الفعلية للنظام.
- **مخططات التنالي التصميمية:** تُعيد صياغة السيناريوهات الأساسية من الفصل السابق، لكن بعد فتح الصندوق الأسود وإظهار التفاعل الفعلي بين المكونات الداخلية.

### 2.3 الأنماط المعمارية والتصميمية

تم تبني مجموعة من الأنماط المعمارية والتصميمية المعيارية لتأمين المرونة اللازمة للنظام وتسهيل صيانته واختباره، ومن أبرز هذه الأنماط:

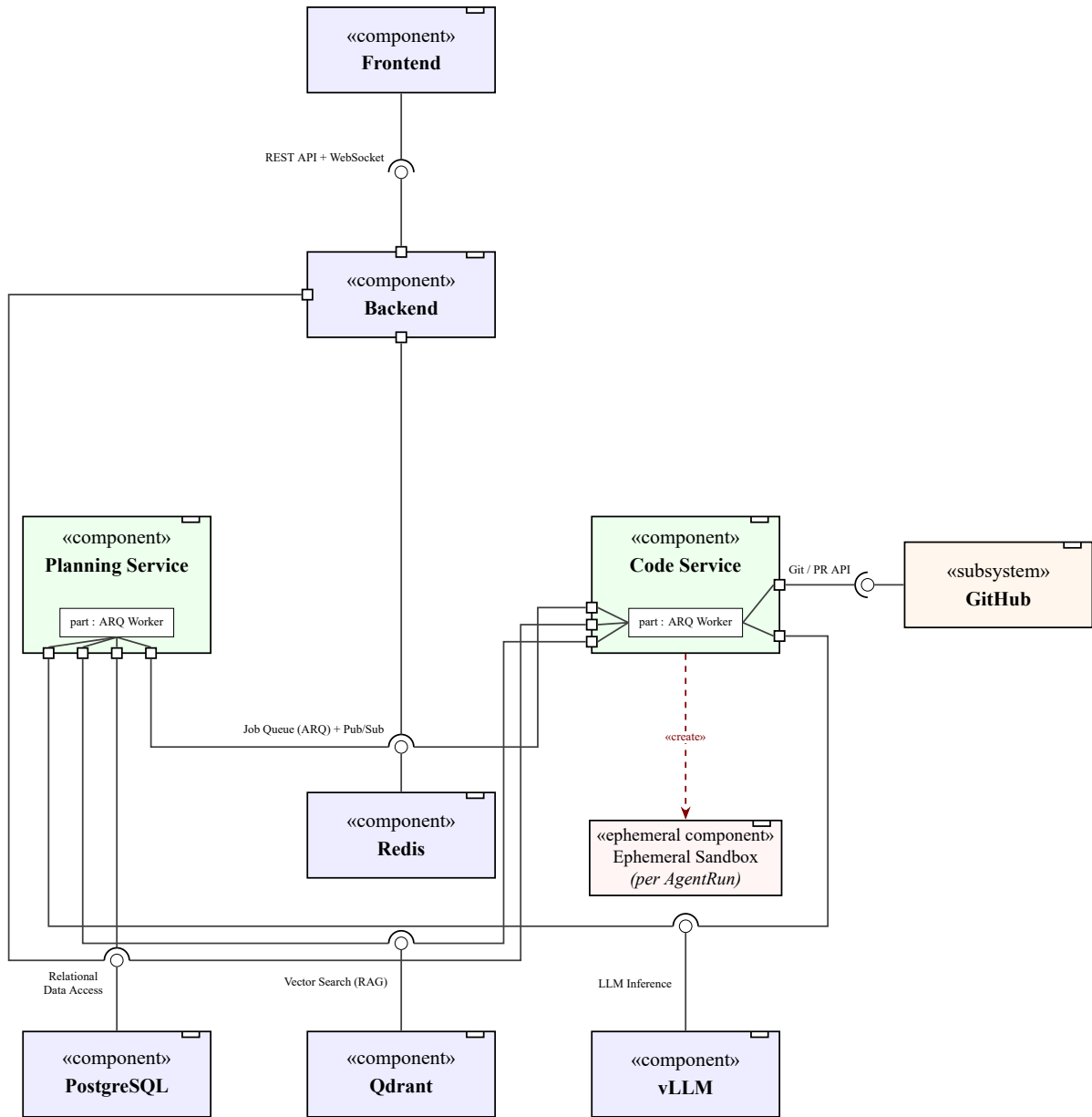
- **Client-Server with two Microservices:** يعتمد النظام في بنيته الهيكلية على نموذج زبون-خادم (Client-Server Architecture)، حيث تمثل الواجهة الأمامية المكون الذي يتفاعل معه المستخدم، وتتواصل مع الـ Backend. يعتمد النظام على خدمتين مصغرتين (Microservices) منفصلتين وظيفياً هما: خدمة التخطيط (Planning Service) وخدمة الترميز (Coding Service).
- **نمط المستودع (Repository Pattern):** يُستخدم لعزل طبقة الـ Data Access عن طبقة الـ Business Logic. من خلال هذا النمط، تتعامل الخدمات مع واجهات مجردة للاستعلام والكتابة دون معرفة التفاصيل الدقيقة لكتابة استعلامات SQL أو هيكلية قاعدة البيانات العلائقية.
- **نمط وحدة العمل (Unit of Work):** يعمل بالتكامل مع نمط المستودع لضمان معالجة المعاملات (Transactions) بشكل ذري (Atomic). يضمن هذا النمط أن جميع العمليات التي تتم على مستودعات متعددة ضمن سياق عمل واحد إما أن تُحفظ معاً في قاعدة البيانات (Commit) أو يتم التراجع عنها بالكامل (Rollback) في حال حدوث أي خلل، مما يحافظ على سلامة واتساق البيانات.
- **Singleton Pattern:** تم تطبيقه على المكونات ذات تكلفة الإنشاء العالية التي يجب مشاركتها عبر كامل دورة حياة الخدمة، مثل اتصال قاعدة البيانات المتجهية Qdrant، ومحرك اتصال قاعدة البيانات PostgreSQL.
- **نمط القفل الموزع (Distributed Lock):** طُبِّق لضمان عدم حدوث تعارضات عند محاولة عدة وكلاء تعديل الملفات نفسها بشكل متزامن. يضمن هذا النمط حصر التعديل على ملف معين بوكيل واحد فقط في اللحظة الزمنية الواحدة، مع توليد رموز حماية متزايدة رتیباً (Fencing Tokens) لإبطال العمليات المتأخرة دلاليّاً.

- بيئة التنفيذ المعزولة (Sandboxing): نمط حماية يُستخدم لعزل عمليات الوكيل البرمجي بالكامل داخل حاوية مستقلة ومقيدة الموارد والصلاحيات، مما يضمن ألا تؤثر الشيفرات البرمجية المولدة أو الأوامر المنفذة من قبل النموذج اللغوي على خادم الاستضافة الأساسي.

### 3.3 مخطط مكونات النظام (Component Diagram)

يوضح الشكل 1.3 مخطط مكونات UML لمنصة Synapse، باستخدام (ball-and-socket notation). يُمثل كل مكون برمجي بمستطيل يحمل الوسم النمطي «component» وأيقونة المكون المعيارية، وتُمثل الواجهة المُقدّمة (Provided Interface) بدائرة صغيرة مصمتة (lollipop) متصلة بخط مباشر بالمكون الذي يقدّمها، بينما تُمثل الواجهة المطلوبة (Required Interface) بنصف دائرة (socket) على حدود المكون المعتمد، بحيث تشير فتحة نصف الدائرة نحو الكرة التي يتصل بها عبر موصل التجميع (assembly connector). لتفادي ازدحام المخطط، لا يتكرر اسم الواجهة عند كل مقيس؛ فالاسم يظهر مرة واحدة بجانب الكرة، ويُستدل على الواجهة المطلوبة من خط الاتصال المؤدي إليها.

- الواجهة الأمامية (Frontend): تعتمد على Backend عبر واجهة موحدة تجمع REST API واتصال WebSocket دائم.
- الخلفية (Backend): تُقدّم واجهة الوصول للواجهة الأمامية، وتعتمد على Redis لإدراج المهام والاشتراك في التحديثات، وعلى PostgreSQL للوصول إلى البيانات (وهي المالكة الوحيدة لترجيحات المخطط عبر Alembic).
- خدمة التخطيط وخدمة الترميز: يحتوي كل منهما داخلياً على جزء (part) من نوع عامل ARQ، وكلاهما يعتمد على نفس واجهات Redis وPostgreSQL (كنسخة SQLAlchemy Core مطابقة دون حق التعديل على البنية)، إضافة إلى Qdrant للاسترجاع المعزّز بالتوليد وvLLM للاستدلال اللغوي. تنفرد خدمة الترميز باعتمادها على GitHub مباشرة، وبإنشائها حاوية معزولة مؤقتة لكل عملية تنفيذ.



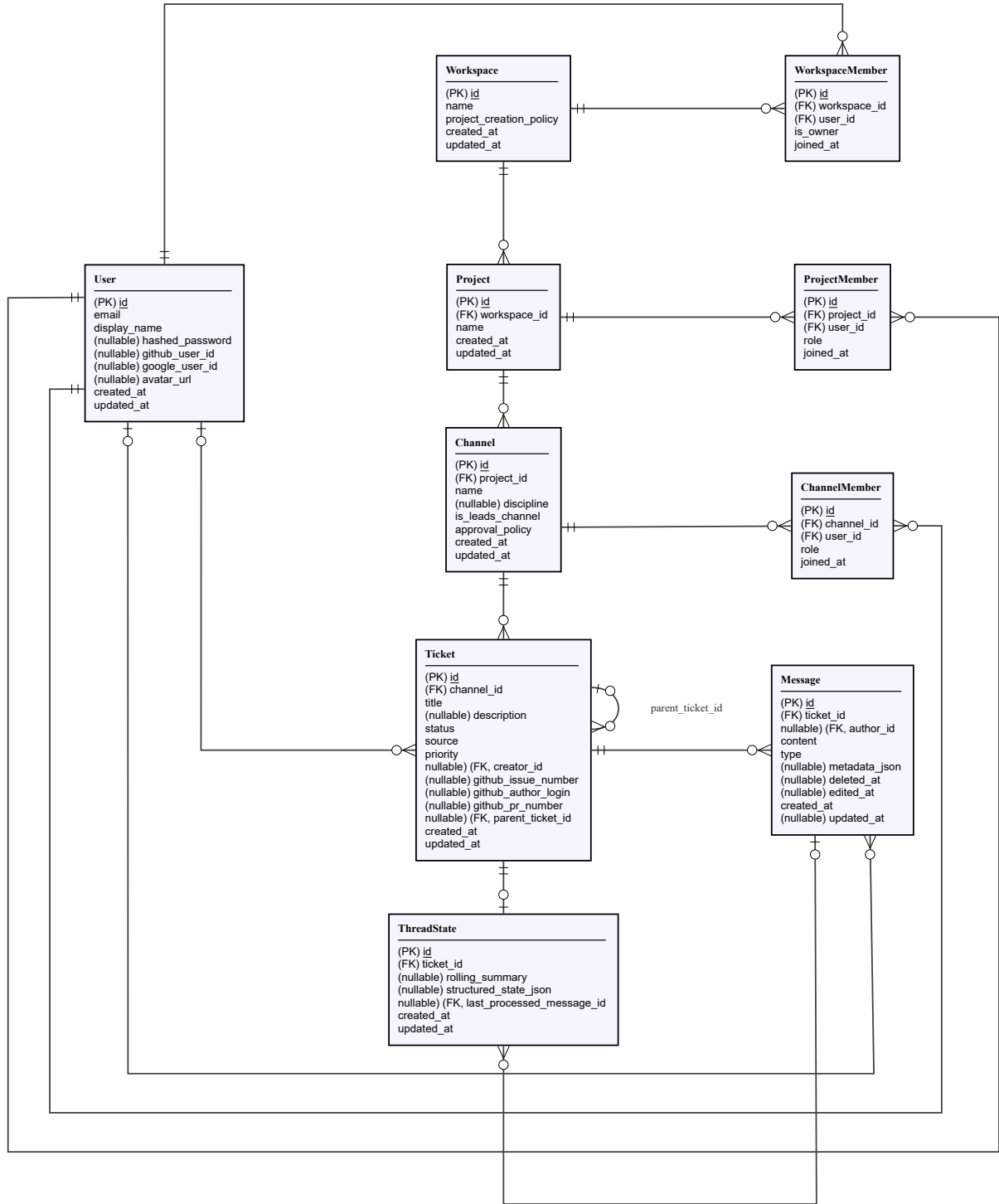
الشكل 1.3: مخطط مكونات النظام (UML Component Diagram)

## 4.3 تصميم قاعدة البيانات (ERD)

نظراً لكبر حجم نموذج البيانات، يُعرض تصميم قاعدة البيانات على مخططين منفصلين لتحسين الوضوح: الأول يغطي كيانات النطاق الأساسي (إدارة المستخدمين والمساحات والتذاكر)، والثاني يغطي كيانات تنفيذ الوكلاء والتكامل الخارجي.

### 1.4.3 كيانات النطاق الأساسي

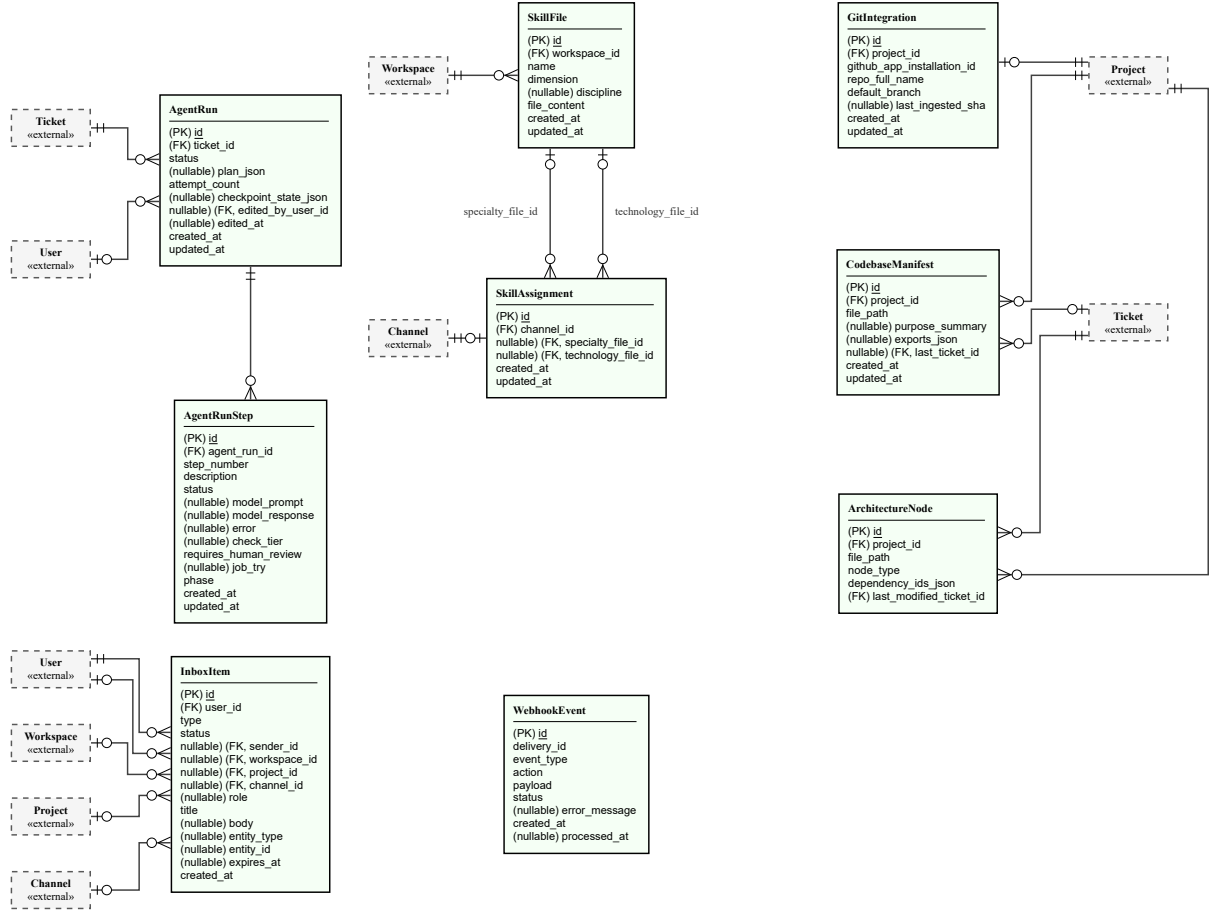
يوضح الشكل 2.3 الكيانات المسؤولة عن التسلسل الهرمي Workspace → Project → Channel → Ticket، وعضوية المستخدمين في مساحات العمل والمشاريع والقنوات، والمراسلة داخل التذاكر.



الشكل 2.3: مخطط الكيانات والعلاقات — النطاق الأساسي

### 2.4.3 كيانات تنفيذ الوكلاء والتكامل الخارجي

يوضح الشكل 3.3 مخطط الكيانات والعلاقات ضمن سياق وكلاء الذكاء الاصطناعي والتكامل مع الأنظمة الخارجية.



الشكل 3.3: مخطط الكيانات والعلاقات — تنفيذ الوكلاء والتكامل

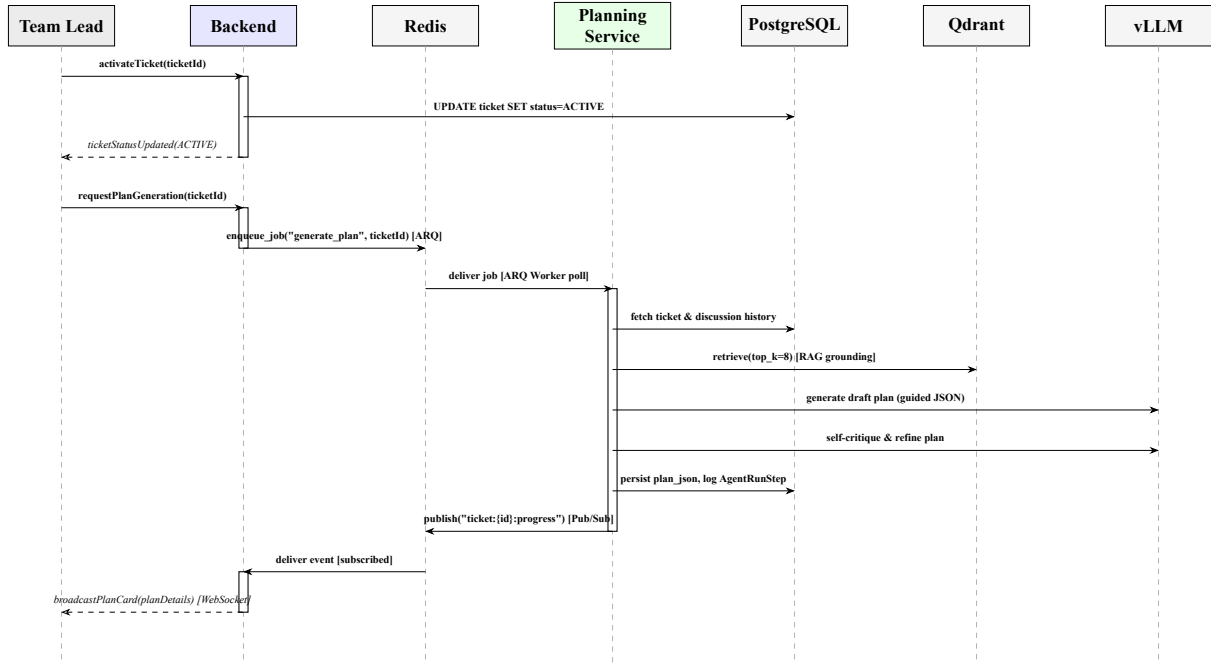
**ملاحظة تصميمية:** الكيان WebhookEvent سجلٌ عام لأحداث GitHub الواردة (بدون علاقة مباشرة بكيانات أخرى)، ويُستخدم حقل `delivery_id` لضمان معالجة كل حدث مرة واحدة فقط (`idempotency`) عند تكرار وصول نفس الحدث من GitHub لمنع حدوث ثغرات التكرار ومعالجة الطلبات المتماثلة في الخوادم الخلفية.

## 5.3 مخططات التتالي التصميمية

يُعيد هذا القسم صياغة السيناريوهين الأساسيين من الفصل السابق (الشكل 2.2 والشكل 3.2)، بعد فتح حدود النظام وإظهار التفاعل الفعلي بين مكوناته الداخلية: Backend، Redis (طابور ARQ وقناة Pub/Sub)، خدمة التخطيط، خدمة الترميز، PostgreSQL، Qdrant، وvLLM. أما بقية سيناريوهات الفصل السابق (إنشاء التذكرة عبر GitHub Issue، إغلاق التذكرة عند الدمج، وربط المشروع بالمستودع) فهي عمليات تُعالج بالكامل داخل Backend دون تفويض لخدمات الوكلاء، وبالتالي فإن تفصيلها لا يضيف مكونات جديدة عما ورد في مخططاتها الأصلية.

### 1.5.3 مخطط التتالي التصميمي: توليد خطة التنفيذ

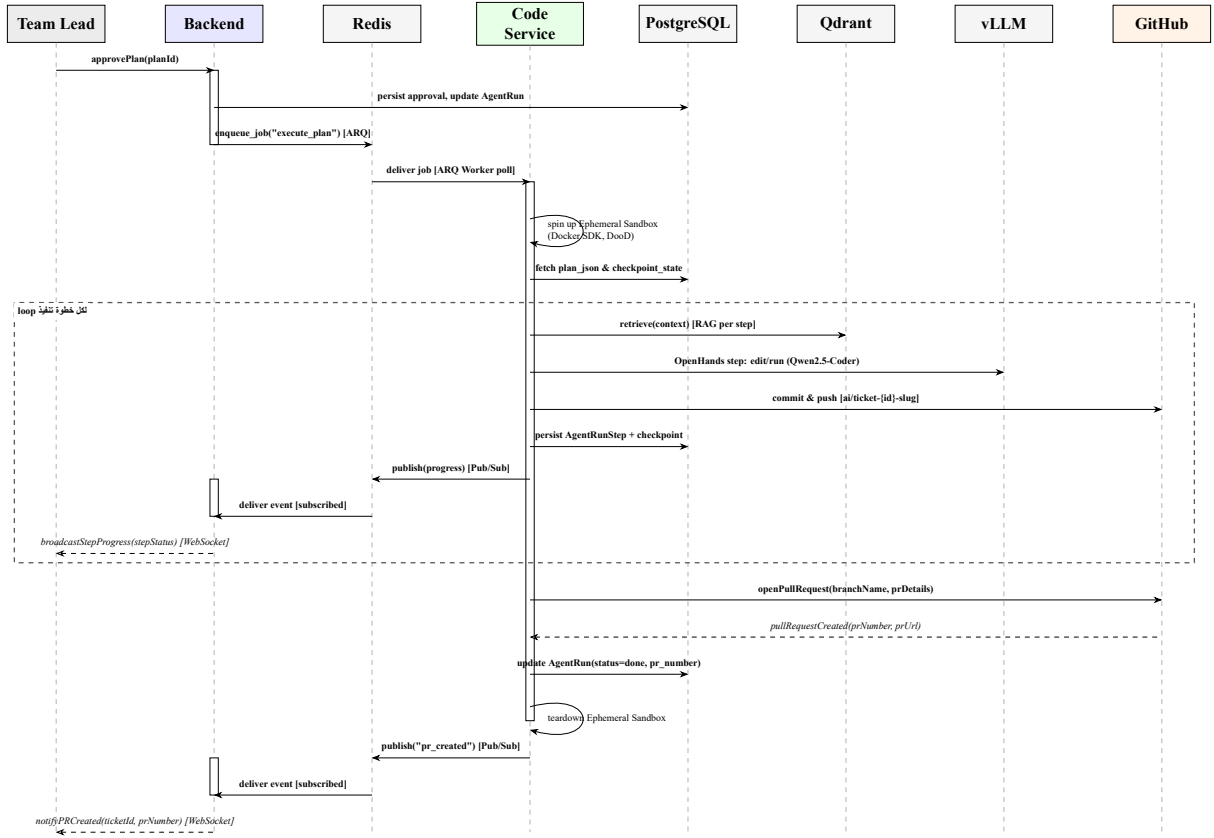
يوضح الشكل 4.3 كيف يُترجم طلب توليد الخطة إلى مهمة غير متزامنة تُنفّذها خدمة التخطيط، اعتماداً على نمط المسودة ثم المراجعة الذاتية (`draft-then-self-critique`) مع الاستعلام عن Qdrant لتثبيت الخطة بالشفيرة الفعلية قبل توليدها عبر النموذج اللغوي.



الشكل 4.3: مخطط التالي التصميمي لعملية توليد خطة التنفيذ

### 2.5.3 مخطط التالي التصميمي: تنفيذ خطة الترميز

يوضح الشكل 5.3 كيف تُنفَّذ الخطة المعتمدة عبر خدمة الترميز خطوةً بخطوة، حيث تُنشأ حاوية معزولة مؤقتة واحدة لكل عملية تنفيذ (AgentRun)، وتُنفَّذ خطوات الخطة ضمن حلقة تكرارية مع ارتكاز (commit/push) بعد كل خطوة، ويُفتح طلب الدمج تلقائياً على GitHub عند اكتمال جميع الخطوات.



الشكل 5.3: مخطط التتالي التصميمي لعملية تنفيذ خطة الترميز

**ملاحظة تصميمية:** تُنفَّذ عمليات Git (الفرع، الالتزام، الدفع، وفتح طلب الدمج) مباشرةً من قبل خدمة الترميز عبر OpenHands Agent SDK، دون المرور بالخلفية كوسيط، بخلاف التمثيل المبسط في مخطط التتالي التحليلي (الشكل 3.2) الذي اعتبر النظام ككل مسؤولاً عن هذه العمليات لأغراض التجريد التحليلي.



# الفصل الرابع

## تنفيذ النظام

### 1.4 مقدمة

بعد الانتهاء من مرحلة التصميم وتحديد البنية الهيكلية ومخططات التفاعل وقاعدة البيانات في الفصل السابق، ينتقل النظام في هذا الفصل إلى حيز التنفيذ الفعلي. يهدف هذا الفصل إلى استعراض الجوانب التطبيقية والبرمجية لمنصة Synapse، وتوضيح كيفية ترجمة المخططات والمواصفات النظرية إلى مكونات برمجية ملموسة تعمل معاً بشكل متكامل ومتناسق. يستعرض الفصل في البداية مراجعة شاملة لبيئة التطوير وحزمة التقنيات والبرمجيات المستخدمة في بناء الواجهات الخلفية والأمامية وخدمات الوكلاء الذكية. وأخيراً، يستعرض الفصل تفاصيل تنفيذ كل مكون من مكونات النظام، مع توضيح آليات المعالجة غير المتزامنة، وحماية بيئة التنفيذ، والتعامل مع حالات الفشل التقني وسلوكيات النماذج اللغوية تكرارياً دون إدراج كتل برمجية مباشرة، صوناً للتوصيف الأكاديمي الرصين.

### 2.4 البيئات والأدوات المستخدمة

تم اختيار حزمة التقنيات المستخدمة في بناء المنصة بعناية لتلبية متطلبات الأداء العالي والمعالجة غير المتزامنة والتكامل السلس مع نماذج الذكاء الصناعي المحلية. يوضح القسم التالي دور كل تقنية في النظام:

#### React 1.2.4

تمثل حجر الأساس للواجهة الأمامية (Frontend) للنظام باستخدام الإصدار 19 مع لغة TypeScript. تم استخدام بيئة بناء Vite لتأمين التطوير السريع والتجميع الفعال، واعتمدت تقنية CSS Modules لعزل التنسيقات البرمجية لكل مكون واجهة بشكل مستقل لمنع تداخل الأنماط. كما تم استخدام مكتبة Zustand لإدارة الحالة العالمية الخفيفة (مثل حالة المستخدم المصادق عليها)، ومكتبة React Query لإدارة ومزامنة حالة البيانات المسترجعة من الخادم الخلفي وإجراء التحديثات الذكية وتفريغ الذاكرة المخفية.

#### FastAPI 2.2.4

إطار العمل المعتمد لبناء الخدمة الخلفية الأساسية (Backend) بلغة Python. تم اختياره نظراً لأدائه العالي المبني على محرك ASGI ودعمه الأصيل لعمليات الإدخال والإخراج غير المتزامنة (async/await). يتيح الإطار تجميع وتوليد توثيق تفاعلي فوري لجميع واجهات البرمجة (OpenAPI/Swagger)، ويقدم نظام حقن تبعيات (Dependency Injection) مرن يُسهّل إدارة جلسات قاعدة البيانات والمصادقة.

#### Visual Studio Code 3.2.4

بيئة التطوير المتكاملة (IDE) الأساسية التي تم استخدامها لكتابة وتطوير الشيفرة البرمجية لجميع خدمات النظام، مدعومة بإضافات تتبع الأخطاء والتنسيق التلقائي للرماز.

#### OpenHands Agent SDK 4.2.4

الإطار البرمجي المعتمد لتمكين وكيل الترميز (Code Agent) من التفاعل مع بيئة العمل. توفر المنصة واجهات برمجية مجردة تسمح للنموذج اللغوي باستخدام أدوات تحرير الملفات، تشغيل الأوامر في محطة التحكم Bash، وتتبع مجريات التنفيذ تكرارياً مع عزل كامل للبيئة.

## 5.2.4 vLLM

محرك استدلال عالي الأداء يُستخدم لخدمة النموذج اللغوي الكبير (LLM) محلياً. يوفر محرك vLLM سرعة استجابة عالية بفضل خوارزميات الانتباه المقسم (PagedAttention) ويقدم واجهة برمجية متوافقة تماماً مع معايير OpenAI، مما يسهل دمجها مع برمجيات النظام، بالإضافة إلى دعم ميزة المخرجات المهيكلة (Structured Outputs) بالاعتماد على مخططات JSON Schema.

## 6.2.4 Postgres

نظام إدارة قواعد البيانات العنصرية المعتمد لحفظ البيانات المستمرة للنظام. تم استخدام الإصدار 17 عبر محرك الاتصال غير المتزامن asyncpg ومكتبة SQLAlchemy. يتولى نظام Postgres حفظ سجلات المستخدمين، والمساحات، والمشاريع، والقنوات، والتذاكر، والرسائل، وسجل خطوات تنفيذ الوكلاء بدقة متناهية. كما تم استخدام أداة Alembic لإدارة وإجراء ترقيات المخطط (Database Migrations) بشكل آمن ومتتابع.

## 7.2.4 Redis

خادم بنية بيانات في الذاكرة يُستخدم لعدة مهام حيوية في النظام: يعمل كوسيط لإدارة طوابير المهام غير المتزامنة لخدمات الوكلاء عبر مكتبة ARQ، ويدير عمليات الأقفال الموزعة للفصل بين جلسات الوكلاء المتزامنة، ويقوم بدور وسيط الرسائل (Message Broker) عبر قنوات الاشتراك والبلث (Pub/Sub) لإرسال الأحداث اللحظية من الخدمات الخلفية إلى قنوات الـ WebSocket الخاصة بالمستخدمين.

## 8.2.4 Qdrant

قاعدة البيانات المتجهية المعتمدة لتخزين أشعة التضمين الخاصة بملفات ومقاطع المشاريع البرمجية. تدعم قاعدة بيانات Qdrant البحث عالي الأداء بالتشابه الدلالي عبر مقياس جيب التمام (Cosine Similarity)، وتم إعدادها لتدعم الفرز متعدد المستأجرين (Tenant Isolation) باستخدام معرف المشروع project\_id كشرط أساسي لضمان أمان وعزل بيانات المشاريع البرمجية المختلفة.

## 9.2.4 Docker

بيئة الحاويات المعتمدة لعزل وتشغيل خدمات النظام وتسهيل نشرها باستخدام أداة Docker Compose. تبرز أهمية Docker القصوى في نظام الوكلاء، حيث يتم إنشاء حاوية برمجية معزولة ومقيدة الموارد بشكل ديناميكي ومؤقت لكل عملية تنفيذ (Ephemeral Sandbox) يطلقها وكيل الترميز لمنع تشغيل أي رماز ضار أو غير آمن خارج حدود البيئة المعزولة.

## 10.2.4 sentence\_transformers

المكتبة البرمجية المستخدمة في دومين التضمين وتوليد الأشعة دلاليًا للرماز البرمجي والنصوص. تم استخدام نموذج التضمين المفتوح nomic-ai/CodeRankEmbed محلياً لتوليد أشعة بأبعاد تبلغ 256 بعداً، مما يضمن دقة عالية في تمثيل المعاني الدلالية لوظائف وتوابع المشاريع البرمجية.

## 11.2.4 Github

منصة استضافة المشاريع الخارجية المعتمدة لإدارة الشيفرة البرمجية. تم بناء تطبيق GitHub App خاص بالمنصة، يرتبط بالخلفية عبر استقبال أحداث الـ Webhooks الموقعة برمجياً لضمان الموثوقية، وإرسال الطلبات إلى واجهة برمجة تطبيقات GitHub REST API باستخدام رموز وصول مؤقتة لإنشاء الفروع وفتح طلبات الدمج (Pull Requests) تلقائياً.

## 3.4 تنفيذ مكونات النظام

### Backend 1.3.4

يتخذ Backend النمط المعماري Modular Monolith من الناحية الكئية، مقسماً داخلياً إلى وحدات نطاق (domain modules) مستقلة منها المصادقة، ومساحات العمل، والمشاريع، والقنوات، والتذاكر، والرسائل، والمهارات، وتكامل GitHub، وسجل تشغيل الوكيل، وصندوق الوارد، والاتصال اللحظي عبر WebSocket. كل وحدة من هذه الوحدات تُبنى وفق نفس البنية الطبقة الأفقية (N-Tier): طبقة توجيه (Router) تستقبل الطلبات، وطبقة خدمة (Service) تحمل منطق العمل، وطبقة مستودع (Repository) للوصول إلى قاعدة البيانات، إضافةً إلى النماذج (Models)، ومخططات التحقق (Schemas)، والتبعيات (Dependencies) الخاصة بكل وحدة. هذا التكرار المتماثل للبنية عبر جميع الوحدات هو ما يجعل Backend قابلاً للتوسع دون أن تتشابك وحداته ببعضها.

**وحدة العمل** فوق طبقة المستودعات تقف وحدة عمل مجردة واحدة تُورث منها وحدة عمل خاصة بكل نطاق. الصنف الأساسي يفرض على كل تنفيذ فرعي منه الالتزام بدورة حياة مدير سياق (context manager) تُنفذ التراجع (rollback) تلقائياً عند خروج غير طبيعي من الكتلة، بينما يبقى الالتزام (commit) فعلاً صريحاً لا يحدث إلا حين تنجح كل العمليات ضمن الكتلة. كل وحدة عمل فرعية لا تضيف شيئاً سوى مجموعة من المستودعات المطبوعة (typed) الخاصة بنطاقها، فيتكرر هذا النمط دون أي تكرار فعلي في الشيفرة.

**الاتصال بقاعدة البيانات والإعداد** يُدار الاتصال بقاعدة PostgreSQL عبر محرك غير متزامن مبني على SQLAlchemy، مزوّد بتجمّع اتصالات ثابت الحجم مع سعة تجاوز إضافية، وفحص دوري لصلاحية الاتصال قبل استخدامه لتفادي أخطاء الاتصالات المينة. تُحقن الجلسة في كل طلب عبر تابع اعتمادية واحد، ويُعيد تلقائياً إلى حالتها الأصلية عبر التراجع إن حدث استثناء أثناء معالجة الطلب. أما الإعدادات الكئية فهو مركزي بالكامل في صنف إعدادات واحد مبني على Pydantic Settings، يُحمل من ملف بيئة واحد، ويضم: رابط قاعدة البيانات، ورابط Redis، ومفتاح توقيع رموز JWT ومدة صلاحية رمز الوصول والتجديد، وبيانات اعتماد كل من تطبيق OAuth الخاصين بـ GitHub و Google، إضافةً إلى بيانات اعتماد تطبيق GitHub المخصص لتكامل المستودعات (وهي مفصلة تماماً عن بيانات OAuth الخاصة بتسجيل الدخول، لأن الأولى تخوّل الوصول إلى الشيفرة بينما الثانية تخوّل الوصول إلى المنصة فقط) بما فيها مفتاح خاص مشفر بترميز Base64 وسرّ التحقق من صحة أحداث Webhook.

**نظام الصلاحيات** لا تُطبّق الأدوار على مستوى مساحة العمل ككل، بل على مستوى كل مشروع على حدة، إلى جانب أدوار أدق على مستوى القناة نفسها (كأن يكون العضو قائداً لقناة تخصصية بعينها). كل عملية حساسة من ربط مستودع Git، إلى الموافقة على خطة، إلى تحرير تذكرة تمر عبر تابع تحقق صريح يفحص الترتيب التالي: هل المستخدم مالك لمساحة العمل؟ ثم هل هو قائد فريق على مستوى المشروع؟ ثم، بحسب العملية، هل هو قائد القناة المعنية تحديداً؟ هذا الترتيب متكرر بحرفيته عبر الوحدات، وهو ما يمنح النظام مجموعة من الصلاحيات المتسقة رغم تعدد نقاط التحقق.

**الاتصال اللحظي** تُبنى طبقة WebSocket فوق آلية نشر/اشتراك (Pub/Sub) في Redis، حيث يُحسب لكل مستخدم متصل المجموعة الدقيقة من مفاتيح البث التي يجب أن يشترك بها: مجرى شخصي خاص به، ومجرى لكل قناة هو عضو صريح فيها، إضافةً إلى كل قناة في كل مشروع يحمل فيه دوراً واسع الصلاحيات (كقائد فريق أو مستشار أو مُشاهد)، وأخيراً لمالكي مساحات العمل كل قناة في كل مشروع تحت أي مساحة عمل يملكونها. هذا الحساب المسبق للاشتراكات هو ما يسمح لثلاث خدمات مستقلة (Backend، وخدمة التخطيط، وخدمة التوليد البرمجي) بنشر أحداثها اللحظية مباشرة على نفس مفاتيح Redis دون المرور عبر Backend كوسيط في كل مرة.

**الوسيط نحو خدمات الذكاء الاصطناعي** يمتلك Backend وحدة مساعدة واحدة مسؤولة حصراً عن إنشاء تجمّع اتصال مشترك بطاير Redis غير المتزامن وإدراج المهام فيه وهي الوحدة الوحيدة التي تعرف أسماء المهام التي تنتظرها خدمات التخطيط والتوليد البرمجي. Backend نفسه لا يستدعي أي نموذج لغوي ولا يتصل بقاعدة Qdrant المتجهية إلا في حالة واحدة استثنائية: عند تحرير خطة تنفيذ يدوياً من قبل قائد الفريق، حيث يستعلم Backend مباشرة عبر واجهة HTTP الخاصة بـ Qdrant للتحقق الميكانيكي من أن الملفات المستهدفة بالتعديل أو الحذف موجودة فعلاً في الفهرسة (وأن الملفات المستهدفة بالإنشاء غير موجودة)، تماماً كما تفعل خدمة التخطيط نفسها، تفادياً لجولة ذهاب وإياب كاملة عبر خدمة التخطيط.

**دورة حياة التذكرة وتحفيز الوكلاء** تحفيز وكيل التخطيط فعل يدوي صريح يقوم به قائد القناة أو قائد الفريق من داخل المحادثة، وليس تلقائياً بأي شكل: يتحقق Backend أولاً من الصلاحية، ثم من عدم وجود عملية تخطيط نشطة بالفعل على نفس التذكرة (يفحص أولي سريع، ثم بالاعتماد على قيد تفرد جزئي على مستوى قاعدة البيانات نفسها يرفض الإدراج الثاني إن وصل الاثنان في اللحظة ذاتها)، ثم ينتقل بحالة التذكرة، وينشئ سجل تشغيل وكيل جديد، ويُدرج مهمة توليد خطة في الطاير. وبالمثل، فإن الموافقة على خطة مولدة

من قبل قائد الفريق هي ما يُدرج مهمة تنفيذ في الطابور المعزول الخاص بخدمة التوليد البرمجي، بينما رفض الخطة يعيد التذكرة إلى حالتها القابلة لإعادة التحفيز، وتحرير الخطة يدوياً يمزّ أولاً عبر نفس آلية التحقق الميكانيكية قبل قبول أي تعديل.

### Frontend 2.3.4

يُبنى تطبيق الواجهة الأمامية باستخدام React 19 مع TypeScript، ويُدار بناؤه وتشغيله التطويري عبر أداة Vite. يعتمد التوجيه بين الصفحات على React Router، بينما يُترجم كل عنصر بصري تقريباً إلى مكون دالي (functional component) منفصل، بما يتسق مع بقية النظام في الفصل بين المسؤوليات.

#### البنية التنظيمية للشيفرة

يتبع رماز الواجهة الأمامية اصطلاح Bulletproof React في تنظيم المجلدات، وهو اصطلاح قائم على تجميع الرماز حول الميزة الوظيفية (feature) لا حول نوع الملف، ويختلف بذلك عن نمط Feature-Sliced Design. فكل ميزة من ميزات النظام كالمصادقة، ومساحات العمل، والمشاريع، والقنوات، والتذاكر، والرسائل، وتشغيلات الوكيل، وتكامل GitHub تُنظّم في مجلد مستقل يضم بدوره طبقات فرعية ثابتة: طبقة استدعاء واجهة برمجة التطبيقات، والمكونات البصرية الخاصة بالميزة، والخطافات (hooks)، والصفحات، وتعريفات الأنواع. هذا الفصل يجعل كل ميزة وحدة شبه مستقلة يمكن تطويرها أو مراجعتها بمعزل عن غيرها.

#### إدارة الحالة

تُفصل حالة التطبيق إلى نوعين بوضوح: حالة الخادم (server state)، وحالة الواجهة المحلية. حالة الخادم كبيانات المستخدم، والتذاكر، والرسائل، وتشغيلات الوكيل تُدار بالكامل عبر TanStack Query، الذي يتولى التخزين المؤقت، وإعادة الجلب، وحالات التحميل والخطأ دون الحاجة لتكرار هذا المنطق يدوياً في كل مكون. أما الحالة المحلية الخفيفة كهوية المستخدم الحالي أثناء الجلسة، أو رسائل التنبيه العابرة فتُدار عبر Zustand. من القرارات المتعمدة هنا أن مخزن المصادقة لا يحتفظ برمز الدخول (token) نفسه؛ إذ يبقى هذا الرمز في ملف تعريف ارتباط محمي من الوصول عبر (httpOnly cookie) تديره طبقة الـ Backend مباشرة، بينما يحتفظ المخزن بكانن المستخدم فقط لتفادي تمرير الخصائص عبر شجرة المكونات.

#### التزامن الفوري

يُفتح اتصال وحيد ودائم عبر WebSocket عند مستوى التطبيق بأكمله، ويُستهل فقط حين يكون المستخدم مسجّل الدخول، مع إعادة محاولة اتصال تلقائية عند الانقطاع غير المتعمد. الجانب الأهم في هذا التصميم أن الأحداث الواردة من الخادم لا تُستخدم لتحديث حالة الواجهة مباشرة؛ بل تُستخدم فقط لإبطال (invalidate) مفاتيح التخزين المؤقت المقابلة في TanStack Query، مما يدفع التطبيق لإعادة جلب البيانات المحدثة من الخادم تلقائياً. بهذا يبقى الخادم مصدر الحقيقة الوحيد، ويُجنّب صنف كامل من الأخطاء الناتجة عن محاولة مزامنة نسختين من الحالة يدوياً بين الخادم والعميل.

#### مكتبة المكونات المشتركة والتنسيق

بدلاً من الاعتماد على مكتبة واجهات جاهزة، بُنيت مجموعة مكونات مشتركة داخلية خفيفة تغطي الاحتياجات المتكررة عبر النظام: مؤشرات التحميل، والشارات (بما في ذلك شارات مخصصة لكل تخصص تقني)، والنوافذ المنبثقة وحوارات التأكيد، ونظام التنبيهات المؤقتة، وحقول الإدخال الأساسية. يمنح هذا الأسلوب تحكماً كاملاً في المظهر وحجم الحزمة النهائية. أما التنسيق البصري فيعتمد على CSS Modules، حيث يرتبط كل مكون بملف أنماط خاص به ذي نطاق معزول، تجنباً لتضارب أسماء الأصناف بين الميزات المختلفة.

#### الحماية والتوجيه

تُفرض قواعد الوصول عند مستوى المسارات عبر مكونين غلافيين: أحدهما يمنع المستخدم غير المسجّل من الوصول للصفحات المحمية ويبعد توجيهه لصفحة الدخول، والآخر يمنع المستخدم المسجّل من العودة لصفحات الدخول والتسجيل. يُقرأ القرار في الحالتين من حالة المصادقة المحلية المذكورة أعلاه دون استدعاء إضافي للخادم.

### Planning Agent 3.3.4

يُنقَد وكيل التخطيط بالكامل داخل خدمة مستقلة تعمل كعامل طابور غير متزامن. يتّبع مسار التنفيذ داخل هذه الخدمة نمط أنابيب ومرشحات (Pipes-and-Filters) بأربع مراحل متتابعة وصريحة: بوابة نطاق، استرجاع، زوج توليد ومراجعة ذاتية، ثم تحقق من

حالة الملفات المُستهدفة في الخطة. مسؤولية تنسيق تتابع هذه المراحل معزولة بالكامل في وحدة واحدة لا تعرف شيئاً عن الطابور أو عن إعادة المحاولة بل تكتفي باستدعاء كل مرحلة بالترتيب وترك أي استثناء يمرّ دون التقاطه.

**بوابة النطاق** قبل أي استرجاع من Qdrant وقيل أي استدعاء توليدي، تمرّ كل تذكّرة عبر استدعاء أولي خفيف للنموذج اللغوي المحلي، بميزانية رمزية صغيرة جداً، يُطلب منه فيه إصدار حكم واحد فقط: هل نص التذكّرة والنقاش المرافق لها قابل للتنفيذ هندسياً أصلاً، أم أنه غامض أو غير قابل للتحويل إلى خطوات برمجية؟ هذا الاستدعاء يُقَدِّد ناتجه عبر آلية فك الترميز الموجّه (guided/structured decoding) الخاصة بـvLLM إلى مخطط ثابت يحمل قيمة منطقية وسبباً موجزاً من جملة واحدة، بدرجة حرارة صفرية لضمان الاتساق. هذه البوابة تحكم فقط على نصّ التذكّرة ذاته، دون أن ترى أي مقاطع مسترجعة أو أي ملفات مهارة، لأنها تصدر حكماً مبدئياً لا حكماً تنفيذياً. رفض البوابة يوقف خط الأنابيب بأكمله فوراً قبل تحمّل أي تكلفة استرجاع أو توليد، وتُعاد التذكّرة إلى حالتها السابقة مع نشر رسالة نظامية تحمل سبب الرفض.

**التوليد المعزّز بالاسترجاع (RAG)** تُفهّز كل مستودعات المشاريع في مجموعة Qdrant واحدة مشتركة على مستوى المنصة بأكملها، وليس مجموعة منفصلة لكل مشروع؛ العزل بين المشاريع لا يتحقق بفصل فيزيائي للمجموعات، بل بحقل حمولة (payload) إلزامي على مستوى المجموعة نفسها، ويشترط كل استعمال استرجاع تصفيته على هوية المشروع. عدد النتائج المسترجعة قابل للضبط عبر متغيّر بيئة.

يعتمد تقسيم الملفات إلى مقاطع (chunking) توزيعاً حسب نوع الملف: ملفات الشيفرة (Python، وJavaScript/TypeScript)، تُقسّم على مستوى الدوال والأصناف عبر أشجار تحليل نحوي مبنية بمكتبة tree-sitter، مع قاعدة خاصة للدوال السهمية المجهولة (arrow functions): تُعامل كوحدة مستقلة فقط إن كانت مسندة صراحة إلى متغيّر مسمّى، أما إن كانت استدعاءً ضمنياً داخل تعبير آخر فلا تُعتبر وحدة قائمة بذاتها. ملفات Markdown تُقسّم عند حدود العناوين الرئيسية. ملفات الإعداد (مثل json، وyaml، وtoml، وملف تركيب حاويات Docker Compose) تُعامل كوحدة واحدة كاملة نظراً لصغر حجمها عادة، باستثناء ملفات القفل (lockfiles) التي تُستبعد كلياً رغم مشاركتها الامتداد مع ملفات إعداد شريعية. تُستبعد أيضاً مجلدات التبعيات المولّدة، ومخرجات البناء، والأصول الثنائية والصور. وأي مقطع يتجاوز الحدّ الرمزي الأقصى لنموذج التضمين يخضع لتقسيم احتياطي بسيط يتراكم سطرًا سطرًا حتى الوصول إلى الحد.

نموذج التضمين المستخدم من عائلة nomic-embed-text، ويحمل داخل عملية العامل نفسها عبر مكتبة sentence-transformers مرة واحدة فقط عند تشغيل الخدمة. النسخة المحدّدة فعلياً هي نسخة مخصّصة لاسترجاع الرّماز مبنية على نفس معمارية nomic-embed-text وتشارك امتدادها الطبيعي للسياق البالغ 8192 رمزاً. يعتمد النموذج بادئات توجيهية غير متماثلة: بادئة للمحتوى المفهّز تختلف عن بادئة الاستعلام، وتُحسب أعداد الرموز بدقة عبر مُجرّئ (tokenizer) النموذج نفسه لا بتقدير تقريبي.

**استيعاب المستودع والمزامنة** يُحتفظ بنسخة محلية واحدة من مستودع كل مشروع تُعاد مزامنتها عبر الزمن بدل استنساخها من جديد في كل مرة؛ الاستنساخ الأول كامل غير مبتور (shallow)، تحديداً لأن الفارق (diff) مقابل (commit) قديمة على كل عملية استيعاب لاحقة يستلزم بقاء تلك الـ (commit) القديمة قابلة للوصول محلياً، وهو أمر لا يوفره استنساخ مبتور. في كل تشغيل لاحق، تُجرى عمليتا جلب ودمج (fetch وpull)، ثم يُحسب فارق بالحالة (إضافة/تعديل/حذف) بين آخر commit تم استيعابها والرأس الحالي، بحيث لا يُعاد تقسيم وتضمين إلا الملفات المتغيّرة فعلاً؛ التشغيل الأول الذي لا توجد فيه commit سابقة يعامل كل ملف متنبّع في المستودع كملف مُضاف حديثاً. التحديثات التراكمية تستبدل نقاط الملف المتغيّر القديمة بنقاط جديدة (حذف ثم إدراج)، بينما الملفات المحذوفة تُزال نقاطها كلياً من الفهرسة، وتُحدّث آخر commit مستوعبة على سجل تكامل Git الخاص بالمشروع فور اكتمال المسح. تُحقّر عملية الاستيعاب هذه إما من مُعالج حدث دفع (push) قادم من Webhook، مصفّى بحيث لا يُستجاب إلا لدفعات الفرع الافتراضي للمستودع تحديداً، أو عند ربط المستودع للمرة الأولى.

**تجميع السياق ومعالجة تجاوز النافذة** يُجمّع سياق كل استدعاء توليدي من محتوى ملفي المهارة (التخصّص والتقنية)، والمقاطع المسترجعة من Qdrant، ورسائل خيط النقاش الخام مرتبة زمنياً. إن تجاوز العدّ الرمزي الدقيق للسياق الميزانية المخصّصة، لا تُحذف أي رسالة من الخيط إطلاقاً، بل يُشغّل حلقة تلخيص تتابعية على طراز "التنقيح" (refine): الدفعة الأولى من الرسائل تُلخّص مباشرة، ثم تُطوى كل دفعة لاحقة بترتيبها الزمني الصارم في الملخّص الجاري تحديثه، دفعة تلو الأخرى، لا دفعة واحدة تضم كل شيء. هذه العملية تُسجّل كخطوة تدقيق مستقلة قائمة بذاتها، ويُعاد بعدها تجميع السياق الكامل مع استبدال الرسائل الخام بالملخّص المضغوط.

**زوج التوليد والمراجعة الذاتية** تُنفّذ الخطة عبر استدعاءين متتاليين للنموذج نفسه: الأول يُنتج مسودة خطة تطوير هيكلية بدرجة حرارة منخفضة لكن غير صفرية (لإبقاء الخطة طبيعية الصياغة دون الانزلاق إلى تكرار حرفي جامد)، والثاني يُعاد فيه تمرير تلك المسودة بأكملها إلى النموذج، إلى جانب السياق الأصلي المجمع، بصياغة صريحة تطلب منه نقدها ومراجعتها وهذه مراجعة ذاتية اتساقية (self-consistency)، لا مصدر تجذّر إضافي. كلا الاستدعاءين يُقَدِّد ناتجهما عبر آلية فك الترميز الموجّه إلى نفس مخطط JSON الثابت الخاص بشكل خطة التطوير، بدل توليد نص حرّ يُؤمّل أن يطابقه مُحلّل لاحق.

**التحقق من حالة الملفات المستهدفة في الخطة** هذا التحقق يقتضي بعدم الاستعانة باستدعاء لغوي ثالث لأجله. يمرّ على كل خطوة في المسوّدة النهائية: إن كان الإجراء تعديلاً أو حذفاً، يُشترط وجود مقطع واحد على الأقل مفهرس بالفعل لمسار الملف المستهدف في Qdrant وإن كان الإجراء إنشاءً، يُشترط ألا يكون الملف موجوداً في الفهرسة أصلاً والخطوات التي لا تُجري أي تغيير يتم تجاوزها. عند أي مخالفة يتم إفشال الخطة بأكملها فوراً دون قبول جزئي ودون حلقة تصحيح تلقائي، وتُعاد التذكّرة إلى حالتها السابقة مع نشر السبب المحدّد للمخالفة.

كل مرحلة من المراحل الأربع أعلاه بما فيها التلخيص عند تفعيله تُسجّل سجلّ تدقيق خاصاً بها بمجرد اكتمالها، بمعزل عن الحالة النهائية التي تنتهي إليها عملية التشغيل ككل وهذا ما يضمن بقاء أثر التدقيق لكل ما تمّ إنجازه فعلاً حتى في حال فشل التشغيل عند مرحلة لاحقة.

**معالجة الإخفاقات** الإخفاقات التقنية الصلبة (كعدم القدرة على الوصول إلى نموذج التوليد) تُعالج عبر آلية إعادة المحاولة الخاصة بالطاير نفسه، بينما رفض بوابة النطاق وفشل تحقّق التجذّر كلتاهما تُعيد التذكّرة فوراً إلى حالة قابلة لإعادة التحفيز، وتُنشر رسالة نظامية توضيحية عبر نفس آلية النشر اللحظي المستخدمة للرسائل العادية، ولا تُعاد محاولتهما أبداً.

**تشغيل النموذج اللغوي فعلياً** تصل كل من خدمة التخطيط وخدمة التوليد البرمجي إلى النموذج اللغوي عبر رابط أساس (base URL) واحد مضبوط بالكامل عبر متغيّر بيئة، موجّه إلى نقطة نهاية متوافقة مع واجهة OpenAI؛ هذا الفصل التام بين الشيفرة وموقع الاستدلال الفعلي هو ما جعل تبديل خلفية الاستدلال ممكناً دون أي تعديل برمجي على الإطلاق. نظراً لقيد العتاد المتاح فعلياً أثناء التطوير (بطاقة رسوميات بذاكرة محدودة لا تكفي لتشغيل نموذج Qwen2.5-Coder-7B-Instruct أو أي نموذج لغوي أضخم المستهدف محلياً عبر vLLM)، استُضيف النموذج فعلياً خلال معظم مراحل التطوير على بيئة تشغيل مجانية توفرّ معالج رسوميات عبر Google Colab: يُشغّل داخل تلك البيئة نفس خادم vLLM المتوافق مع واجهة OpenAI، ثم يُكشف عن منفذه للعمامة مؤقتاً عبر نفق Cloudflare ضبط رابط الأساس في متغيّر البيئة على عنوان ذلك النفق مباشرة. لم يتغيّر سطر واحد من رماز أي من الخدمتين نتيجة هذا الترتيب والفارق الوحيد بينه وبين نشر النموذج محلياً بالكامل هو قيمة متغيّر بيئة واحد.

#### 4.3.4 Coding Agent

المُنسّق المركزي لتنفيذ الخطة المعتمدة هو حلقة خطوات واحدة تقود المستودع، والحاوية المعزولة، والفعل المورّع، ومحادثة OpenHands، والتحقّق، وحلقة التصحيح، وكشف الحلقات العالقة، والتثبيت والدفع (Commit and Push) بعد كل خطوة، وتحديث فهرس المستودع، وفتح طلب السحب، وهو أيضاً المسؤول الوحيد عن هدم الحاوية في نهاية المطاف. عند بدء كل محاولة تشغيل، تُحمّل أولاً أرقام الخطوات التي اكتملت فعلاً في محاولات سابقة من قاعدة البيانات، وتُحذف أي سجلات خطوات بقيت عالقة في حالة "قيد التنفيذ" بسبب تعطل سابق؛ فالمحاولة المُستأنفة لا تترث أبداً سجلاً نصف مكتمل، وتتخطى مباشرة كل خطوة سبق أن نجحت.

**تحضير المستودع** يُستنسَخ المستودع من جديد في كل محاولة تشغيل إلى مجلد عمل خاص بها على المضيف، ثم إمّا يُسحب فرع بعينه موجود مسبقاً، أو يُنشأ فرع جديد تماماً من الفرع الافتراضي للمشروع. اصطلاح التسمية يشقّ من عنوان التذكّرة نصّاً مبسطاً (slug) مضافاً إلى معرّف التذكّرة.

**دورة حياة الحاوية المعزولة** تتحدّث خدمة التوليد البرمجي مباشرة إلى Docker daemon المضيف عبر المقبس (socket) الممرّر إليها، بدل تشغيل daemon متداخل خاص بها (نمط Docker-outside-of-Docker). تُنشأ حاوية معزولة واحدة بالضبط لكل تشغيل وكيل، وتبقى حيّة طوال مدة ذلك التشغيل بأكمله ولا يُوصّل داخلها بأي مسار من المضيف سوى مجلد العمل الخاص بهذا التشغيل تحديداً، بغضّ النظر عمّا قد يحاوله الوكيل وهو ضمان أقوى من محاولة اعتراض الأفعال الخطرة في مجرى الأحداث، لأن حدود التوصيل (mount boundary) لا يمكن الالتفاف عليها منطقياً مهما كانت المحاولة. تُشغّل الحاوية خادم عميل OpenHands مضمناً في صورتها منذ لحظة بنائها، ويُنشر منفذه إلى منفذ حرّ يختاره Docker ديناميكياً لا منفذ ثابت أبداً بما أن تشغيلات متزامنة عدّة تستلزم كل منها منفذها الخاص دون تصادم. يكتشف خط التوليد البرمجي وقت التشغيل ما إذا كانت حاويته وحاوية الحاوية المعزولة "شقيقتين" على نفس شبكة Docker؛ إن كانتا كذلك، يُوجّه الاتصال مباشرة إلى عنوان الحاوية الشقيقة الداخلي، وإلا فيُستخدم عنوان الاسترجاع المحلي كخيار احتياطي. تُنشأ الحاويات بصلاحيات نظام مجرّدة بالكامل، ومنع صريح لتصعيد الامتيازات، وسقف محسوب للمعالج والذاكرة.

**تدرّج الموارد** يُطبّق نموذج طبقتين (طبقة "بسيطة" وأخرى "أثقل" لمشاريع أضخم)، يُختار بينهما بإشارة حتمية، على غرار الفصل الذي يعتمده Kubernetes بين الطلب (request) والحدّ الفعلي (limit): الطبقة المطلوبة تُحدّ بهامش أمان من الموارد الحرة فعلياً على المضيف في لحظة الطلب (لا يُمنح أي تشغيل أكثر من ثلاثة أرباع ما هو متاح فعلياً، لإبقاء هامش للمضيف نفسه ولأي تشغيل متزامن آخر)؛ وإن كان حتى هذا التخصيص المخفّض دون حدّ أدنى للجدوى العملية، تُرفض الحاوية المعزولة صراحة كإخفاق تقني صلب، لا كتخصيص أقل صمتاً.

**تكامل حزمة OpenHands** عملية العامل (worker process) في خدمة التوليد البرمجي تلعب دور "المتحكم" في هذه الحزمة: تبني كائن الوكيل (النموذج، وقائمة الأدوات) محلياً، لكن تنفيذ الأدوات نفسه يحدث داخل خادم العميل المضمن في الحاوية المعزولة، ولا يتم الوصول إليه إلا عبر WebSocket/HTTP قبل فتح أي محادثة، يستطلع المتحكم نقطة فحص صحة خادم العميل داخل الحاوية بشكل دوري حتى مهلة محدّدة؛ وعند انتهاء المهلة دون استجابة صحيحة، يُستخرج آخر ما وصلت إليه حالة الحاوية ومقتطف من سجلاتها ويُرفق بالخطأ لتشخيص السبب بدل الفشل الصامت.

محادثة واحدة طويلة العمر تُفتح لكل تشغيل وكيل وكل خطوة في الخطة تُسلّم إلى نفس هذه المحادثة كرسالة جديدة، بحيث تستمر سلسلة استدلال النموذج عبر الخطوات المتعاقبة بدل إعادة تصفيرها في كل مرة. ولأن استدعاءات التشغيل والإرسال في هذه الحزمة متزامنة (synchronous) بطبيعتها، يُنفذ كل استدعاء لها من حلقة الخطوات غير المتزامنة على خيط (Thread) تنفيذ خلفي منفصل، تفادياً لتجميد حلقة الأحداث الخاصة بالعامل بأكملها وهو ما كان سيوقف كل مهمة أخرى تعالجها نفس العملية طوال مدة الخطوة. مجموعة الأدوات المستخدم هي مجموعة الأدوات الافتراضية للحزمة مع مكثف سياق يبقي المحادثة المترابطة ضمن الميزانية مع تقدّم الخطوات.

### حراس التنفيذ الثلاثة

- **القفل الديناميكي:** كلما لمس الوكيل ملفاً لم يكن ضمن النطاق المُعلن أصلاً للخطوة الجارية، يُطلب قفل ذلك المسار فوراً ويُضاف إلى مجموعة الأقفال المحفوظ بها لتلك الخطوة؛ وفشل الحصول على القفل يطلب إيقاف التشغيل بدل هدم الحاوية مباشرة.
- **كشف الحلقة العالقة:** تُشتق بصمات SHA-256 من اسم الأداة المستدعاة، ووسائطها المسلسلة، ومقتطف من نتيجتها، وتُتابع ضمن نافذة منزلفة محدودة الحجم لأخر عدد من الاستدعاءات؛ إن تكررت البصمة نفسها فوق عتبة معيّنة داخل النافذة، يُحقن تحذير في المحادثة الحية يطلب من النموذج تجربة نهج مختلف؛ فإن استمر تكرار البصمة نفسها بعد التحذير، يُحسب كل تكرار لاحق ضد ميزانية استنفاد منفصلة، وعند نفاذها يُوقف التشغيل ويُرفع عائق يحمل تفاصيل البصمة كدليل.
- **تنبيه المسارات المحمية:** أي خطوة تلمس ملفات ضمن مجلد ترحيل قاعدة بيانات، أو مسار/اسم إعدادات تكامل مستمر معروف (بما يشمل صراحة إعدادات خطوط أتمتة النشر، لأن حقن تعليمات لم تُحدّد بالكامل عبر حدّ المحتوى غير الموثوق ينبغي ألا يتمكّن من الخروج عن نطاق التذكرة الفعلي عبر تعطيل الفحوصات ذاتها التي قد تكشفه) يُعلم لمراجعة بشرية إلزامية؛ هذا الحارس وحده لا يطلب إيقاف التشغيل ولا يلمس الحاوية إطلاقاً، بل يكفي بوسم سجل الخطوة.

**التحقّق المرحلي متعدد المستويات** بعد كل دورة (أو دورات) للوكيل ضمن خطوة، يُقرّر النجاح أو الفشل عبر تدرّج من ثلاث طبقات: أولاً ما يُعلنه المستودع نفسه (أهداف الفحص والاختبار في ملف Makefile، أو نصوص مماثلة في بيان حزمة Node.js إن وُجد)؛ فإن لم يوجد أيّ منهما، أو أعلن كلاهما صراحة غياب الهدف المطلوب، يُنتقل إلى طبقة ثانية من فحوصات عامة لكل امتداد ملف (فحص وتنسيق HTML و CSS و Python، وفحص صياغة فقط لملفات JavaScript)؛ فإن لم تنطبق أي منها أيضاً، تُستخدم طبقة أخيرة من فحوصات سلامة أساسية بحتة تتحقّق فقط من كون الملفات الملموسة نصّاً صالحاً بترميز UTF-8، وخالياً من بايتات فارغة أو علامات تعارض لم تُحلّ. الخطوة التي كانت الخطة تتوقّع فيها صراحة فعلاً يغيّر ملفات، لكن لم يلمس أي ملف فعلياً على القرص، تُعتبر إخفاً صريحاً عند أول طبقة فحص، لا نجاحاً صامتاً لأن هذا النمط عادة ما يعني أن الوكيل وصف التغيير فقط دون تنفيذه فعلياً.

**حلقة التصحيح عند الإخفاق الخفيف (soft failure)** عند فشل التحقّق، يُعاد المحتوى الحالي للملفات الملموسة مع سجلّ الفشل الخام إلى النموذج كرسالة جديدة ضمن نفس المحادثة، ضمن ميزانية محدودة من محاولات التصحيح حيث المحاولة الأولى تتلقّى سجلّ الفشل الأخير فقط، بينما كل محاولة تالية تتلقّى التاريخ التراكمي الكامل لكل إخفاق سابق في نفس الخطوة، لا آخر إخفاق فقط، حتى يتمكّن النموذج من ملاحظة تكراره للخطأ نفسه إن حدث. استنفاد الميزانية دون نجاح يُصنّف كإخفاق منطقي منفصل.

**انضباط التثبيت والدفع** كل خطوة تُغيّر شيئاً على القرص تثبّت وتُدفع إلى فرع التذكرة فور اجتيازها التحقّق مباشرة، لا في نهاية التشغيل مجمعة؛ والدفع لا يُفرض قسراً (force push) إطلاقاً. رفض الدفع بسبب عدم التقدّم السريع (شيء آخر لمس نفس الفرع المملوك للوكيل) لا يُحلّ تلقائياً بسحب ودمج، بل يُصعد فوراً كفة عائق مستقلة مرفقة بسجلّ التثبيتات (Commits) المتباعدة الفعلي كدليل، لأن التوفيق بين فرع لمسه طرف آخر قرار بشري لا حكم آلي.

**تحديث فهرس المستودع تراكمياً** بعد دفع كل خطوة ناجحة، يخضع كل ملف لمسته تلك الخطوة لإعادة فهرسة هيكلية (تحليل بنية شجرة تجريد النحو الخاصة بلغة Python للملفات المكتوبة بها مع رجوع إلى مسح بمكتبة tree-sitter لبقية اللغات المدعومة، لاستخراج معالم مثل تعريفات الدوال والأصناف ومحدّات CSS إلى جانب جملة موجزة يولّدها النموذج المحلي نفسه) ويُدرج السجل أو يُحدّث (أو يُحدف كلياً للملفات المزالة) في نفس جدول الفهرس المشترك الذي يستعين به وكيل التخطيط في فحص التعارض الدلالي. أي فشل في هذه الخطوة يُسجل فقط دون إسقاط التشغيل بأكمله، لأنها تحديث تكميلي لا بوابة صحة.

**القفْل الموزَّع والتزامن** يُنفَّذ القفل على نسخة Redis واحدة فقط، باستخدام عملية ذرية وليس عبر خوارزمية Redlock هذا تبسيط متعمد وموثق مناسب لبيئة تعتمد نسخة Redis واحدة، بدل أي قفل من جانب العميل، لأن حاويتين معزولتين لا تتشاركان شيئاً على مستوى نظام التشغيل وتحتاجان حكماً خارجياً مرئياً للطرفين.

قبل بدء أي خطوة، تُفحص الملفات المستهدفة المُعلَّنة لها مقابل مجموعة الملفات المسجَّلة لكل تشغيل نشط آخر فإن لم يوجد تقاطع فعلي يُفحص أيضاً مقابل كل تشغيل نشط آخر تعارض دلالي محتمل عبر استدعاء تصنيف بسيط بدرجة حرارة صفريّة للنموذج اللغوي، يُسأل فيه إن كان وصفا المهمتين يعالجان نفس الغرض الأساسي رغم اختلاف الملفات الملموسة وهذا يلتقط تعارضاً قد تسببه خطوتان مستقلتان تلمسان شيفرة مختلفة نصياً لكنها مترابطة دلاليّاً. تُكتسب الأقفال بترتيب أبجدي صارم لمسارات الملفات تحديداً لمنع الجمود (deadlock) بين تشغيلين يحتاجان نفس الملفين لكن بترتيب معكوس؛ وإن فشل أي قفل واحد ضمن دفعة، تترجّع كل الأقفال المكتسبة سابقاً في نفس الدفعة قبل رفع الفشل. كل قفل يحمل مهلة انتهاء ثابتة، مضبوطة عمداً أطول من المهلة الزمنية القصوى المسموحة للخطوة الواحدة، بحيث لا تنتهي صلاحية قفل خطوة لا تزال قيد التنفيذ فعلياً من تحتها. فوق آلية الاستبعاد المتبادل البسيطة هذه، يحمل كل اكتساب قفل أيضاً قيمة عدّاد متزايدة أحاديّاً (monotonic) تُستخرج عبر عملية زيادة ذرية في Redis نفسها "رمز سياج" (fencing token) يُقصد به السماح لدالة تحقّق مساعدة بالتأكد من أن رمز القفل المخزّن لم يتجاوز تشغيل أحدث قبل السماح بأي عملية كتابة. الملفات التي تلمس أثناء الخطوة نفسها ولم تكن ضمن نطاقها المُعلن أصلاً تُقفل ديناميكياً عبر نفس حارس القفل الديناميكي، وتُضاف إلى المجموعة المحتفظ بها، وتُحرّر جميعها معاً عند انتهاء الخطوة (نجاحاً أو فشلاً) وتحرير القفل مضمون دوماً عبر الكتلة الختامية المحيطة بحلقة الخطوة، بصرف النظر عن المسار الذي خرجت منه الخطوة.

**تصعيد الإخفاق النهائي** أي استثناء غير مصنّف يصل إلى غلاف مهمّة الطابور يُعامل كإخفاق تقني صلب، وتُطبّق عليه نفس جدولة إعادة المحاولة المشتركة بين الخدمتين (ثلاث محاولات، بمهلة تراجع متصاعدة تقريباً دقيقة ثم خمس ثم خمس عشرة دقيقة)؛ وعند استفاد هذه الميزانية، تُعلم كل من التذكّرة وسجل التشغيل بالفشل، ويُنشر عائق يحمل عدد المحاولات المستنفدة وآخر خطأ رُصد، بدل ترك التذكّرة عالقّة صامتة.

**فتح طلب السحب وإغلاق الحلقة** بعد اكتمال كل خطوات الخطة بنجاح، يُجمّع وصف طلب سحب هيكلي من عنوان التذكّرة ووصفها إضافة إلى وصف كل خطوة مكتملة والطبقة التي اجتازت التحقّق من خلالها، ويُفتح طلب السحب عبر مُهيئ مزوّد Git المحلول مستهدفاً الفرع الافتراضي للمشروع من فرع التذكّرة؛ يُكتب رقم طلب السحب الناتج مباشرة على سجل التذكّرة، وتنتقل حالتها إلى المراجعة، وتُنشر رسالة إتمام غنيّة تحمل رابط الطلب إلى الخيط. هدم الحاوية المعزولة يحدث كآخر فعل على الإطلاق، داخل الكتلة الختامية الخارجية، بصرف النظر عن أي من كل المسارات أعلاه انتهت إليه التشغيل فعلياً.

## 5.3.4 Async Redis Queue

اسم مكتبة الطابور المستخدمة نفسها arq اختصار لعبارة Async Redis Queue، وهو بالضبط الدور الذي تلعبه هنا: طابور مهام مبني فوق Redis بواجهة غير متزامنة. تم اختيار arq تحديداً لأنه غير متزامن: دوال معالجة المهام نفسها دوال async، تُنفَّذ مباشرة على حلقة أحداث العامل دون أي غلاف تزامني وسيط، ولا يستلزم أي بنية تحتية إضافية تتجاوز Redis المؤرّر أصلاً للنشر/الاشتراك والقفل الموزَّع.

**طابوران معزولان، لا طابور واحد** تستمع خدمة التخطيط إلى طابور arq الافتراضي، بينما تُسمّى خدمة التوليد البرمجي طابوراً خاصاً بها معزولاً صراحة تحديداً لأن مهام تنفيذها طويلة نسبياً (قد تمتد دقائق كاملة لتشغيل OpenHands متعدد الخطوات داخل حاوية معزولة) ولا يجوز أن تتنافس على سعة عامل مع مهام خدمة التخطيط الأقصر أمداً، والعكس صحيح. المُساعد الذي يُدرج المهام في Backend يوجّه ببساطة كل مهمة إلى الطابور المسمّى المناسب بحسب نوعها.

**أنواع المهام وحمولاتها** تعبر هذه الحدود ثلاثة أنواع من المهام: مهمة استيعاب مستودع تحمل معرف مشروع واحداً فقط (تُدْرَج إمّا من مُعالج حدث دفع مصقّى على الفرع الافتراضي، أو بعد ربط مستودع للمرة الأولى)، ومهمة توليد خطة تحمل معرف تذكّرة ومعرف سجل تشغيل الوكيل معاً (تُدْرَج عند تحفيز قائد قناة أو قائد فريق لوكيل التخطيط يدوياً من خيط التذكّرة)، ومهمة تنفيذ خطة تحمل معرف سجل تشغيل الوكيل فقط (تُدْرَج عند موافقة قائد فريق على خطة مودّعة، مستهدفة تحديداً الطابور المعزول لخدمة التوليد البرمجي).

**دورة حياة إقلاع العامل وإيقافه** كل عملية عامل لكل من الخدمتين تُحمّل مسبقاً الموارد المشتركة المكلفة مرة واحدة فقط عند الإقلاع، لا عند كل مهمة: تجمّع اتصال محرك قاعدة البيانات، وأوزان نموذج التضمين (لخدمة التخطيط تحديداً)، وعمل HTTP مجمع موجه مسبقاً إلى نقطة نهاية النموذج اللغوي مع مُجرّئه الرمزي، وتُحرّر جميعها عند الإيقاف. هذا التحميل المسبق هو ما يبقى تكلفة استدعاء كل مهمة على حدة منخفضة، بما أن تكلفة الإعداد لا تُدفع في كل مرة.



**انضباط إعادة المحاولة والتراجع** تتشارك الخدمتان نفس شكل إعادة المحاولة: ثلاث محاولات محدودة بجدول تراجع ثابت (تقريباً دقيقة، ثم خمس دقائق، ثم خمس عشرة دقيقة) لأي إخفاق يُصنّف تقنياً صلباً، تُرفع عبر آلية إعادة المحاولة الخاصة بـ arq نفسه بدل أن تعيد المهمة إدراج نفسها يدوياً؛ ورقم المحاولة الحالي الذي توفره arq لكل استدعاء مهمة يُحفظ أيضاً على السجل المقابل في قاعدة البيانات، بحيث يبقى عدد المحاولات مرئياً للتدقيق وناجياً من أي إعادة تشغيل للعملية نفسها. أما الإخفاقات المنطقية فلا تُعاد محاولتها عبر هذه الآلية إطلاقاً وهي أصلاً نتيجة حكم مُتخذ بالفعل (تذكرة خارج النطاق، أو إيقاف بسبب حلقة عالقة أو تعارض قفل، أو استنفاد ميزانية تصحيح خفيف)، وإعادة محاولتها ستنتج الحكم نفسه فحسب؛ لذا يُرفع الاستثناء مباشرة بدل آلية إعادة المحاولة، فتنتهي المهمة فوراً دون أي محاولة مجدولة إضافية.

**ضبط التزامن** تحدّد كل عملية عامل عدد المهام التي تُشغلها بالتزامن، والمدة القصوى التي يُسمح فيها لمهمة واحدة بالتنفيذ قبل معاملتها كمنتهية المهلة، وكلاهما قابل للضبط عبر متغيّر بيئة خاص بكل خدمة على حدة، انعكاساً لفارق مدد المهام الحقيقي بين الخدمتين حيث أن مهام التخطيط محكومة بزمن استدعاءات النموذج اللغوي وتنتهي في وقت أقصر بكثير من مهام التنفيذ التي تحتاج هامشاً يكفي لتشغيل كامل متعدد الخطوات داخل حاوية معزولة.

**النشر/الاشتراك فوق نفس نسخة Redis** بمعزل عن طابور المهام نفسه، تحمل نفس نسخة Redis أيضاً نقل الأحداث اللحظية لطبقة WebSocket بأكملها؛ وكل واحدة من الخدمات الثلاث تنشر أحداث التقدّم والحالة اللحظية مباشرة على نفس المفاتيح المخصصة لكل قناة، دون توجيهها عبر Backend كوسيط أولاً وهذا ما يسمح للواجهة بمشاهدة تقدّم الوكيل لحظياً وقت حدوثه فعلياً، لا بعد أن يستطلع الـ Backend لاحقاً. من الجدير بالذكر أن النشر عبر هذا المسار في كل من مهمّتي التخطيط والتنفيذ يحدث دوماً بعد إتمام التثبيت (commit) بالكتلة المعاملاتية المرتبطة به، لا قبله ولا داخله؛ فالحالة النهائية تُكتب أولاً وتُغلق كتلتها، ثم يُنشر الإشعار بحيث لا يصل أي مستهلك للحدث اللحظي إلى حالة لم تُلتزم بها قاعدة البيانات بعد.

## 6.3.4 Github Integration

### Repository Linking

يُبنى تكامل Git على نموذج تطبيق GitHub App مخصص، لا على رموز وصول شخصية (Personal Access Tokens) ولا على تطبيق OAuth وحده؛ هوية المنصة (تسجيل الدخول) منفصلة تماماً عن صلاحية الوصول إلى الشيفرة، وربط مستودع بمشروع يتحقق عبر تثبيت هذا التطبيق المخصص، بحيث لا يُحفظ عن الربط سوى معرف عملية التثبيت واسم المستودع الكامل وفرعه الافتراضي.

**تدفق التثبيت** يطلب قائد فريق أو مالك مساحة عمل رابط تثبيت مرتبط بمشروعه حيث يُولد رمز حالة عشوائي قصير الأمد ويُخزن في Redis ليضع دقائق محدودة، مربوطاً بالمشروع ويُبنى رابط التثبيت الفعلي بحيث يحمل هذا الرمز كما أن الرابط ذاته لا يشير إلا إلى صفحة تثبيت التطبيق على GitHub، ثم يعيد GitHub التوجيه إلى نقطة استدعاء عودة (callback) بعد إتمام التثبيت من طرف المستخدم.

**تحليل استدعاء العودة** يُحلّ المشروع أولاً عبر رمز الحالة المسترجع من Redis (يُستهلك مرة واحدة فقط ثم يُحذف فور قراءته)، مع بديل احتياطي عبر ملف تعريف ارتباط (cookie) إن لم تنتج رحلة رمز الحالة (كاختلاف سياق المتصفح مثلاً) ثم يُستعلم عن كل مستودع منحه هذا التثبيت صلاحية وصول عبر نقطة نهاية سرد المستودعات الخاصة بالتطبيق، ويُختار "أفضل مرشح" آلياً: يُفضل مستودع غير مرتبط بأي مشروع بعد، ثم احتياطياً مستودع مرتبط أصلاً بنفس المشروع الجاري إعداده تحديداً، وأخيراً كمالذ أخير فقط أول مستودع في القائمة المُعادة؛ وإن تعذر تحديد المشروع نفسه عند هذه النقطة، يفشل التدفق برسالة واضحة تفيد باحتمال انتهاء صلاحية جلسة الإعداد.

**طبقة تجريد مزود Git** لا شيء مما سبق يلمس حمولة GitHub الخام مباشرة؛ منطق GitHub المحدد (توليد رمز هوية موقع للتطبيق نفسه، ومبادلاته برموز وصول قصير الأمد خاص بعملية تثبيت بعينها، واستدعاء سرد المستودعات، وبناء رابط التثبيت) محصور بالكامل خلف واجهة واحدة تُنفذ مرة واحدة لكل مزود؛ وكل واحدة من الخدمات الثلاث — Backend، وخدمة التخطيط، وخدمة التوليد البرمجي — تُعرف نفس هذه الواجهة وتنفيذها الخاص بـ GitHub بشكل مستقل، لأن كل خدمة تحتاجها لغرض مختلف (Backend للربط ومعالجة أحداث Webhook، وخدمة التخطيط للاستنساخ أثناء الاستيعاب، وخدمة التوليد البرمجي للاستنساخ وفتح طلب السحب). إضافة مزود ثانٍ لاحقاً يعني كتابة صنف مُهيئ واحد جديد لكل خدمة وتسجيله، دون أي تعديل في أي موضع آخر يستهلك الواجهة أصلاً.

**استيعاب أحداث Webhook والتحقق منها** صحة كل طلب Webhook وارد يتم التحقق منه عبر توقيع HMAC-SHA256 محسوب على جسم الطلب الخام بالضبط، باستخدام سرّ Webhook المشترك، ويُقارَن بترويسة التوقيع التي يرسلها GitHub، عبر مقارنة ثابتة الزمن تحديداً حيث أن أي طلب يفشل هذا التحقق يتم رفضه فوراً قبل أن يُوثق بأي شيء من حمولته.

**معالجة غير متزامنة وقابلة للتكرار** بمجرد التحقق من الحدث، لا يُعالج مباشرة ضمن نفس طلب HTTP؛ يُفحص أولاً معرف التسليم الخاص به الصادر عن GitHub مقابل التسليمات المخزنة سابقاً (تسليم مكرّر وهو سلوك طبيعي متوقع من GitHub نفسه عند إعادة الإرسال حيث يُنهي بصمت دون معالجة، لا خطأ)، ولا يُخزن بحالة انتظار ولا يُسلم للمعالجة إلا تسليم جديد فعلياً كما أن المعالجة الفعلية للـ Business logic تحدث كمرور ثانٍ منفصل على ذلك السجل المخزن، ينقله عبر حالة "قيد المعالجة" ثم إما "معالج" أو "فاشل" مع رسالة الخطأ المحددة بحيث يترك أي تعطل أثناء المعالجة أثراً واضحاً قابلاً للتفتيش، لا حدثاً مفقوداً صامتاً.

**كشف نقطة نهاية Webhook محلياً عبر ngrok** بما أن الـ Backend كان يعمل خلال معظم مراحل البناء على جهاز تطوير محلي لا خلف نطاق عام، بينما يستلزم تسليم أحداث Webhook من GitHub رابطاً يمكنه الوصول إليه عبر الإنترنت العام، استُخدمت أداة ngrok لفتح نفق HTTPS عام مؤقت يُحوّل مباشرة إلى المنفذ المحلي الذي يعمل عليه الـ Backend عنوان التحويل الصادر عن هذا النفق هو ما سُجّل فعلياً كـ رابط Webhook ورابط استدعاء العودة في إعدادات التطبيق على GitHub. هذا حلّ خاص ببيئة التطوير فقط، ناتج عن عدم توفر نشر عام دائم للـ Backend بعد؛ ورمز معالجة Webhook نفسها لا تدرك ولا تعتمد بأي شكل على كون الطلب قد وصل عبر نفق بدلاً من رابط عام "حقيقي" مباشرة.

**أثر كل حدث وارد** فتح مشكلة (issue) جديدة على المستودع المرتبط ينشئ تذكرة جديدة في القناة المخصصة للقادة في المشروع (حاملة رقم المشكلة الأصلية واسم صاحبها للنتبع)، مع إشعار كل عضو في تلك القناة؛ إعادة فتح مشكلة سبق إغلاقها تُعيد ربط تذكرتها الموجودة أصلاً بنفس قناة القادة، وتُفرغ أي حالة نقاش متراكمة أصبحت غير ذات صلة، وتُعيدّها إلى حالتها الأولى في دورة حياة التذكرة؛ حدث الدفع لا يُستجاب له إطلاقاً إلا إن طابق مرجعه الفرع الافتراضي المضبوط للمستودع تحديداً (أي دفع على فرع آخر يُنجاهل عمداً في هذه الطبقة، لأن إعادة الفهرسة تهتم بالفرع الافتراضي فقط)، وعند التطابق يُدرج طلب استيعاب مستودع في الطابور

بدل تنفيذ أي عمل تضمنين مباشرة داخل معالج الحدث نفسه؛ وإغلاق طلب سحب يُحدّث حالة تذكّره المرتبطة بشكل مختلف بحسب ما إذا كان قد دُمج فعلاً أو أُغلق دون دمج، مع نشر رسالة نظامية مقابلة في الحاليتين.

## Coding Agent Pull Requests

بعد وصف الآلية الكاملة لتنفيذ الخطة في القسم السابق، يركّز هذا الجزء تحديداً على الجانب المتّصل بـ GitHub فعلياً من عملية فتح طلب السحب. فتح طلب السحب يستلزم رمز وصول تثبيت جديداً (بنفس آلية الاستخراج الموصوفة أعلاه)، ويُستدعى تابع فتح طلب السحب الخاص بمُهيئ المزود والمرتبّط مع فرع التذكّرة كمصدر (head)، والفرع الافتراضي للمشروع كهدف (base)، ووصف طلب السحب الهيكلي المجمع. حقلاً الرقم والرابط من الحمولة المُعادة يتم قراءتهما عموماً بالاسم دون افتراض أي شيء آخر عن شكل استجابة مزود بعينه بحيث تُصبح مسؤولية معالجة استجابة مزود مستقبلي مختلفة الشكل إلى نفس هذين الحقلين مسؤولية المُهيئ نفسه، لا مسؤولية الجهة المستدعية.

**توقيت الفتح** بما أن التنبيهات تصل إلى فرع التذكّرة تدريجياً بعد كل خطوة، قيل فتح طلب السحب نفسه بوقت طويل، فإن طلب السحب لا يفتح فعلياً إلا مرة واحدة، كآخر فعل في تشغيل ناجح بالكامل بعد أن تجتاز كل خطوة التحقق وتُدفع لا أن يُنشأ فارغاً مسبقاً ثم يُملأ لاحقاً وهذا ما يمنع أي تشغيل لا يزال قيد التنفيذ أو انتهى إلى عائق من إظهار طلب سحب غير مكتمل قد يصادفه مراجع بشري.

**تركيب وصف طلب السحب** يُجمع الوصف بالكامل من بيانات مسجّلة فعلياً وبشكل دائم أثناء التنفيذ عنوان التذكّرة ووصفها، إضافة إلى وصف كل خطوة اكتملت فعلياً وأي طبقة من طبقات التحقق الثلاث اجتازتها بدل أن يُطلب من النموذج صياغة نص الوصف بشكل منفصل وهذا يُبقي الوصف صادقاً تماماً أثناء التشغيل.

**إغلاق الحلقة لاحقاً** رقم طلب السحب الناتج يُكتب مباشرة على سجل التذكّرة، وتنتقل حالة التذكّرة إلى حالة المراجعة؛ ومن تلك اللحظة فصاعداً، يتولّى مسار Webhook الوارد القياسي والموصوف في القسم الفرعي السابق الأمر بالكامل حيث أن حدث إغلاق لاحق (سواء بالدمج أو دونه) هو ما يُغلق التذكّرة أخيراً أو يعيد فتحها، بحيث تُدار دورة حياة طلب السحب بعد لحظة فتحه بالكامل عبر نفس خط أنابيب Webhook الوارد العام، لا عبر أي استطلاع (polling) خاص بخدمة التوليد البرمجي.

# الفصل الخامس

## خطة اختبار النظام

### 1.5 مقدمة

بعد استعراض تفاصيل تنفيذ مكونات النظام الثلاثة في الفصل السابق، يتناول هذا الفصل الاستراتيجية المعتمدة للتحقق من صحة هذا التنفيذ. تتخذ منصة Synapse قراراً معمارياً بمنح وكيل ذكاء صناعي صلاحية الكتابة الفعلية في مستودعات المستخدمين، ضمن حدود مُحكّمة تُنفّذها آليات القفل الموزّع وحواجز الحماية الموصوفة سابقاً؛ وهذا القرار تحديداً هو ما يرفع كلفة أي خلل غير مرصود في الطبقة المسؤولة عن هذه الحدود من مجرّد عطل وظيفي عادي إلى خطر فعلي على سلامة الرّماز البرمجي للمستخدم. مبدأ "الإنسان يتداول، والوكيل ينفّذ، والإنسان يراجع" الذي تقوم عليه المنصة بأكملها يفترض ضمناً أن كل ما يسبق بوابة المراجعة البشرية النهائية (طلب السحب على GitHub) قد سلك مساراً حتمياً وصحيحاً؛ وخطة الاختبار الموصوفة في هذا الفصل هي الآلية التي تُثبت هذا الافتراض بدل تركه اعتقاداً غير مُتحقّق منه.

تُبنى الخطة على محورين مستقلّين لكنهما متكاملان: محور مستوى العزل، الذي يميّز بين اختبارات الوحدة الضيقة النطاق واختبارات التكامل الأوسع؛ ومحور الانضباط الزمني، الذي يصف اختبار الانحدار كممارسة مستمرة تتكرّر مع كل إضافة جديدة إلى قاعدة الاختبارات، لا كمستوى ثالث مستقل عن الأول. يستعرض الفصل كلا المحورين بالتفصيل، ثم يوضّح كيف طُبّق معاً على الخدمات الثلاث لمنصة Synapse.

### 2.5 استراتيجية الاختبار العامة

تعتمد المنصة نموذج هرم الاختبار (Test Pyramid) [1] إطاراً عاماً لتوزيع الجهد بين مستويات الاختبار المختلفة: قاعدة عريضة من اختبارات وحدة سريعة وكثيرة العدد، تتحقّق من منطق معزول تماماً عن أي بنية تحتية فعلية؛ وطبقة أضيق فوقها من اختبارات تكامل أقل عدداً لكنها أعلى تكلفة وأبطأ تنفيذاً، تتحقّق من أن الحدود بين المكونات المختلفة (خدمة وقاعدة بيانات، أو خدمة وخدمة أخرى عبر طابور مهام) تتصرّف كما هو مفترَض فعلياً، لا كما تفترضه محاكاة داخلية للاختبار وحدها.

الاختبار بين محاكاة كاملة (mock) وبين نسخة مزيفة (fake) تحاكي السلوك الحقيقي لواجهة بعينها هو تمييز جوهري تعتمد المنصة عملياً، منقولاً عن الممارسة الموصوفة في [2]: كل مستودع (Repository) ووحدة عمل (Unit of Work) لهما نسخة مزيفة كاملة تنفّذ نفس الواجهة المجرّدة التي ينفّذها المستودع الحقيقي، بذاكرة داخلية بسيطة بدل اتصال فعلي بقاعدة بيانات؛ هذا يضمن أن اختبار طبقة الخدمة يتحقّق من منطق الخدمة نفسه دون أن يتحمّل كلفة قاعدة بيانات فعلية، ودون أن ينزلق إلى اختبار تفاصيل تنفيذ داخلية غير ذات صلة كما يحدث غالباً مع محاكاة عميقة لكل استدعاء على حدة. تبقى المحاكاة المباشرة (AsyncMock) محصورة في الحدود الخارجية الفعلية للنظام؛ أي حيث لا توجد نسخة مزيفة معقولة أصلاً (عميل HTTP خارجي، أو حزمة SDK تابعة لطرف ثالث)، لا في التعامل بين مكونات النظام الداخلية نفسها.

اختبار الانحدار، بخلاف المستويين السابقين، ليس درجة إضافية في هذا الهرم بل ممارسة أفقية تطبّق عبره بأكمله: في كل مرة تُضاف حالة اختبار جديدة إلى أيّ من المستويين، تُعاد كل الحالات السابقة التي كانت تعمل فعلاً قبل هذه الإضافة، لا للتأكد من صحة الجديد فحسب بل للتأكد من أن إضافته لم تكسر شيئاً كان يعمل مسبقاً بصمت. يفصّل القسم 5.5 هذه الممارسة تحديداً.

### 3.5 اختبارات الوحدة

#### 1.3.5 المنهجية

يتبع كل ملف اختبار وحدة في المنصة بنية ثابتة: تصنيف Fake لكل مستودع ووحدة عمل يفي بالواجهة المجرّدة نفسها التي يفي بها التنفيذ الحقيقي؛ ودوال مساعدة (helpers) تبني كائنات نطاق صالحة لمرحلة التحضير (arrange) في كل حالة اختبار دون تكرار نفس منطق البناء في كل ملف؛ وفصل صارم لكل طبقة معمارية داخل الوحدة (خدمة، توجيه، مستودع/وحدة عمل) في ملفها الخاص، بحيث يشير فشل اختبار واحد إلى طبقة واحدة بعينها لا إلى الوحدة بأكملها بشكل غامض. تعمل كل حالات الاختبار على وضعية غير متزامنة بالكامل (asyncio) انسجاماً مع طبيعة الشيفرة قيد الاختبار نفسها، ويطابق مجلد اختبارات كل وحدة برمجية بنية الوحدة نفسها المكوّنة من طبقاتها الأفقية.

### 2.3.5 النطاق المخطط له عبر الخدمات الثلاث

**الـ Backend** يُعَمِّم النمط الثلاثي المطبق بالفعل بشكل شبه كامل على وحدة المصادقة (طبقة خدمة، وطبقة توجيه، وطبقة مستودع/وحدة عمل، ثلاثتها مغطاة) على كل وحدة نطاق أخرى في النظام: مساحات العمل، والمشاريع، والقنوات، وصندوق الوارد، والتذاكر، وسجل تشغيل الوكيل، وتكامل GitHub، والرسائل، والمهارات، وحالة الخيط، والاتصال اللحظي عبر WebSocket. يُضاف إلى ذلك اختبار طبقة تجريد مزود Git نفسها من زاوية توافقها مع الواجهة المجردة الموحدة، بمعزل عن أي استدعاء فعلي لواجهة برمجة GitHub.

**خدمة التوليد البرمجي** تستهدف اختبارات الوحدة هنا كل قطعة منطق قابلة للعزل التام عن أي حاوية معزولة فعلية أو اتصال Redis حقيقي: المنطق الداخلي لحراس التنفيذ الثلاثة (القفل الديناميكي، وكشف الحلقة العالقة عبر بصمات SHA-256، وتنبيه المسارات المحمية) بمعزل عن أي قفل فعلي، ومحدث فهرس المستودع، وتدرج التحقق المرحلي متعدد المستويات، وتوجيه مُهيئ مزود Git عبر المصنع الخاص به.

**خدمة التخطيط** يستهدف هذا المستوى منطق زوج التوليد والمراجعة الذاتية، وتحقق مخططات فك الترميز الموجّه، وقواعد تقسيم الملفات إلى مقاطع بجميع حالاتها الحديثة (قاعدة الدوال السهمية المجهولة، واستبعاد ملفات القفل، والتقسيم الاحتياطي عند تجاوز الحد الرمزي)، ومنطق حساب الفارق أثناء مزامنة المستودع، وتتابع مراحل خط الأنابيب بما فيها بوابة النطاق. في جميع هذه الحالات، يُستبدل الاستدعاء الفعلي للنموذج اللغوي المحلي بنسخة ثابتة (stub) تُعيد ناتجاً مهيكلًا محدداً سلفاً، بحيث تكون الصحة قيد الاختبار هي صحة منطق التنسيق المحيط بالاستدعاء، لا جودة استجابة النموذج نفسها.

## 4.5 اختبارات التكامل

### 1.4.5 لماذا بنية تحتية فعلية لا محاكاة كاملة

كانت كل اختبارات المنصة، قبل هذه المرحلة، اختبارات وحدة صرفة؛ لا وجود لأي اختبار يمرّ فعلياً عبر قاعدة بيانات، أو نسخة Redis، أو قاعدة بيانات متجهية حقيقية. هذا كافٍ تماماً لمنطق الأعمال المعزول، لكنّه غير كافٍ إطلاقاً لإثبات صحة الطبقة الأكثر حساسية في خدمة التوليد البرمجي: القفل الموزّع والحراس الثلاثة. فنسخة مزيّفة من قفل Redis تعمل داخل نفس العملية وحلقة الأحداث التي يعمل ضمنها الاختبار نفسه، ولا يمكنها منطقياً محاكاة تسابق فعلي بين عاملين مستقلّين حقيقيين على نفس المفتاح؛ إثبات أن آلية الاستبعاد المتبادل تمنع فعلاً الكتابة المزدوجة يستلزم عاملين فعليّين يتنافسان على نسخة Redis حقيقية واحدة، لا محاكاة داخل عملية واحدة. الأمر ذاته ينطبق، بدرجة أقلّ حدّة، على قاعدة PostgreSQL: القيود المرجعية، وأنواع الأعمدة مثل JSONB، والقيد الجزئي الفريد المستخدم لمنع تكرار عملية تخطيط نشطة على نفس التذكرة، سلوكها الفعلي يخصّ محرّك قاعدة البيانات الحقيقي نفسه، ولا تحاكيه بديل خفيف داخل الذاكرة بأمانة كافية.

لهذا السبب تحديداً، تعتمد طبقة اختبارات التكامل الجديدة نسخاً فعلية ومؤقتة (ephemeral) من نفس محرّكات البنية التحتية المستخدمة في بيئة الإنتاج ذاتها (قاعدة PostgreSQL العلائقية، ونسخة Redis، وقاعدة Qdrant المتجهية) عبر أداة Testcontainers [3]، التي تُقلع كل حاوية من هذه الحاويات مرة واحدة فقط عند بدء جلسة الاختبار وتهدمها تلقائياً عند انتهائها. هذا لا يُدخل تبعية تشغيلية جديدة على المشروع؛ فاستخدام Docker أصلاً متطلّب أساسي لا غنى عنه في زمن التشغيل الفعلي (الحاويات المعزولة المؤقتة لخدمة التوليد البرمجي)، لا شيئاً أضيف خصيصاً لأجل الاختبار وحده.

### 2.4.5 الحدود المتعمّدة: ما يبقى محاكى حتى داخل اختبارات التكامل

حتى على هذا المستوى، يبقى الاستدعاء الفعلي للنموذج اللغوي المحلي عبر vLLM، وحلقة تنفيذ الوكيل الكاملة عبر OpenHands، محاكيّين بنسخة ثابتة، لأن كليهما يتطلّب فعلياً عتاد رسوميّات غير متوفّر ضمن بيئة تشغيل اختبار آلية. ما تُثبته اختبارات التكامل الموصوفة هنا تحديداً هو أن منطق التنسيق المحيط بهذين الاستدعاءين — القفل، والحراس، وانتقالات الحالة، وتوجيه المهام عبر الطابور، وطبقة تجريد المزود — يتصرّف بشكل صحيح مقابل البنية التحتية الحقيقية التي يعتمد عليها فعلياً، لا أن ناتج النموذج نفسه جيّد؛ ف جودة الناتج التوليدي مسألة تقييمية منفصلة تماماً تخرج عن نطاق هذا الفصل.

### 3.4.5 سيناريوهات التكامل الأساسية

فيما يلي أبرز السيناريوهات المستهدفة، موزّعة على الخدمات الثلاث:

- دورة حياة تذكرة كاملة، من الإنشاء إلى التفعيل إلى توليد خطة إلى الموافقة عليها، مُنفّذة مقابل قاعدة بيانات فعلية تربط بين وحدات التذاكر والقنوات والمشاريع وسجل تشغيل الوكيل معاً، بدل اختبار كل وحدة بمعزل تام عن الأخرى كما يحدث حالياً.

- استقبال حدث Webhook من GitHub حتى إنشاء تذكرة مقابلة، بما في ذلك التحقق من أن إعادة تسليم الحدث نفسه لا تُنشئ تذكرة مكررة، عبر القيد الفريد الفعلي على معرف التسليم في قاعدة البيانات الحقيقية.
- التحقق من عمود المزود المضاف حديثاً إلى جدول تكامل Git: كتابة وقراءة فعليتان عبر قاعدة بيانات مُرحّلة (migrated) فعلياً، لا افترض أن الترحيل صحيح فقط لأنه طُبّق دون خطأ ظاهر.
- تسابق فعلي بين طلبتي تفعيل أو توليد خطة متزامنين على نفس التذكرة، للتأكد من أن القيد الجزئي الفريد على مستوى قاعدة البيانات، لا الفحص الأولي السريع وحده، هو ما يمنع فعلياً عملية تخطيط مزدوجة.
- عاملان فعليّان (لا محاكاة داخل عملية واحدة) يتنافسان على قفل Redis نفسه لنفس مسار الملف، للتحقق المباشر من ضمان الاستبعاد المتبادل.
- التسابق بين اكتمال محادثة الوكيل وحدث الإيقاف، للتأكد من أن هدم الحاوية المعزولة يحدث مرّة واحدة بالضبط من الكتلة الختامية للمنسّق، بصرف النظر عن أيهما سبق الآخر فعلياً.
- دورة استيعاب كاملة للتوليد المعزّز بالاسترجاع: تقسيم إلى مقاطع، فتضمين، فكتابة فعلية في Qdrant، فاسترجاع مصقّى بهوية المشروع، للتأكد من أن العزل بين المشاريع يعمل فعلياً على مستوى الحمولة لا نظرياً فقط.
- بوابة النطاق كمرحلة أولى في خط الأنابيب، مقابل كتابة فعلية لنتيجتها وانتقال حالة التذكرة إلى حالة الرفض خارج النطاق، عبر قاعدة بيانات حقيقية مرّحلة بنفس الترحيل المستخدم فعلياً لهذه الميزة.

## 5.5 اختبار الانحدار

اختبار الانحدار، بتعريفه المعياري [4]، هو إعادة تشغيل الجزء من قاعدة الاختبارات كان ناجحاً فعلاً قبل تغيير جديد، للتأكد من أن هذا التغيير لم يكسر سلوكاً كان مُتحققاً من صحته سابقاً بصمت؛ فالتحقق من أن السلوك الجديد نفسه يعمل، دون التحقق أيضاً من أن كل ما سبقه لا يزال يعمل، إثبات ناقص. تعتمد المنصة هذه الممارسة كإلزام لطرقة بنائها التراكمية والتدرجية نفسها: بما أن قاعدة الاختبارات تُبنى وحدة بوحدة ونقطة تحقق (checkpoint) بعد أخرى، لا تُعتمد حالة اختبار جديدة إلا بعد إعادة تشغيل كل قاعدة الاختبارات المتراكمة حتى تلك اللحظة بأكملها، لا الحالة الجديدة وحدها معزولة عن سياقها؛ بحيث لا يمكن لحالة اختبار جديدة ناجحة أن تُخفي وراءها انحداراً صامتاً أحدثته في مكان آخر من النظام دون أن يُلاحظ.

عملياً، تُفصل حالات الاختبار عبر علامة تصنيف مخصصة تميّز بين المجموعة الفرعية السريعة القائمة كلياً على المحاكاة والنسخ المزيّفة، والتي تعمل دون أي بنية تحتية خارجية وتُشغّل عند كل تعديل مهما صغر كحلقة تغذية راجعة محلية ضيقة؛ وبين قاعدة الاختبارات الكاملة التي تضم أيضاً طبقة اختبارات التكامل المعتمدة على الحاويات الفعلية، والتي تُشغّل كاملة عند كل نقطة تحقق أكبر قبل اعتبار مرحلة بناء بأكملها منتهية. تجدر الإشارة إلى أن هذا الانضباط، في مرحلة المشروع الحالية، ممارسة مفروضة يدوياً من المطور نفسه لا بوابة آلية مفروضة عبر خط تكامل مستمر (Continuous Integration) مُعدّ سلفاً؛ وربط هذا الانضباط بخط تكامل مستمر فعلي يبقى امتداداً طبيعياً منطقياً لهذا العمل، لا جزءاً منه في صورته الحالية.

## المراجع

- [1] M. Cohn, *Succeeding with Agile: Software Development Using Scrum*. Addison-Wesley Professional, 2009.
- [2] H. Percival and B. Gregory, *Architecture Patterns with Python: Enabling Test-Driven Development, Domain-Driven Design, and Event-Driven Microservices*. Sebastopol, CA: O'Reilly Media, 2020.
- [3] Testcontainers, ``Testcontainers: Integration testing using real dependencies in docker containers," 2024. Official documentation. Accessed: 2026.
- [4] G. J. Myers, C. Sandler, and T. Badgett, *The Art of Software Testing*. Wiley, 3rd ed., 2011.